

정답이 따로 없는 과제

오토인코더 1일차

이틀 뒤의 과제 — 미션 5

- 손상된 문서 이미지를 원본에 가깝게 되돌리기
- 손상 원인 예: 습기 · 찢김 · 얼룩 (미션 안내)
- 모델을 직접 설계하고 학습시켜 노트북으로 제출

손상된 문서 — 합성 예시

damaged (synthetic example)

by that office summary value and in be
paper at on filter method of is image an
data data sample value on method letter
from be sample sample filter summary
the to figure section this for be and
value by paper that number paper of by

clean original

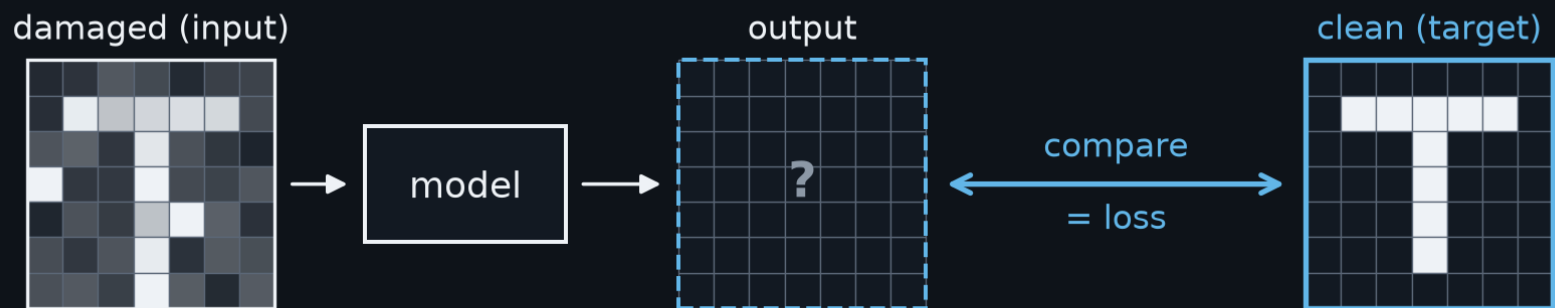
by that office summary value and in be
paper at on filter method of is image an
data data sample value on method letter
from be sample sample filter summary
the to figure section this for be and
value by paper that number paper of by

- 설명용으로 만든 합성 이미지 · 미션 5 실제 데이터 아님
- 왼쪽 damaged · 오른쪽 clean original

질문 — 짝이 있다면

- 손상된 문서와, 같은 문서의 깨끗한 원본이 짝으로 있음
- 이 짝으로 모델은 무엇을 배울 수 있을까

짜이 있으면

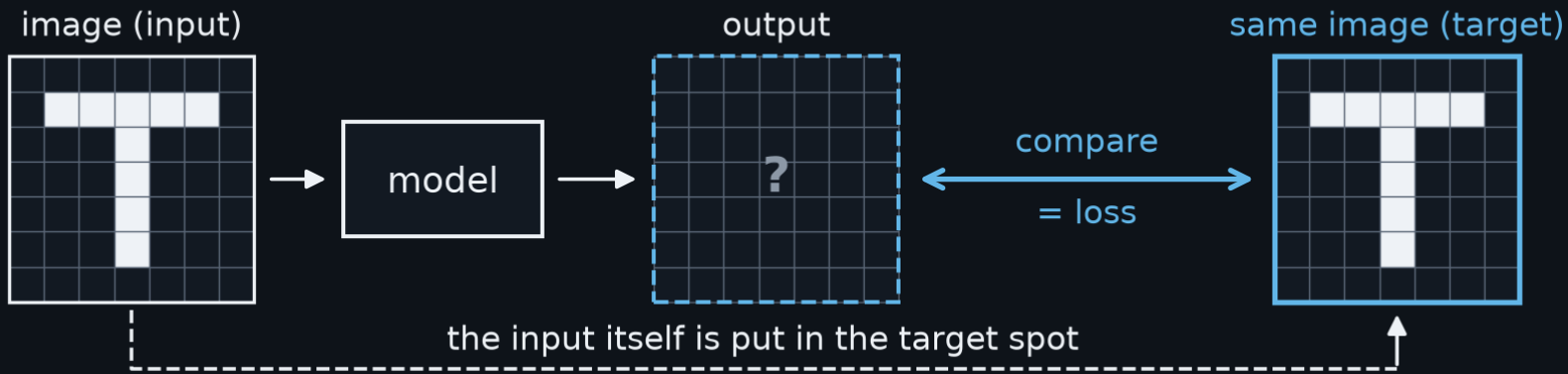


- 입력: 손상된 문서 · 정답: 깨끗한 원본 · 출력과 정답의 차이를 줄이는 학습
- CNN · RNN 강의에서 본 학습 루프와 같은 모양
- 깨끗한 원본을 늘 구할 수 있는 것은 아님

질문 — 라벨이 아예 없다면

- 이미지만 잔뜩 있고, 사람이 붙인 정답은 없음
- 학습 루프의 손실 줄에서 출력과 무엇을 비교할까

정답을 입력 자신으로



- 사람이 붙이는 라벨이 필요 없음
- 손실: 출력과 입력의 차이 · 예: MSE

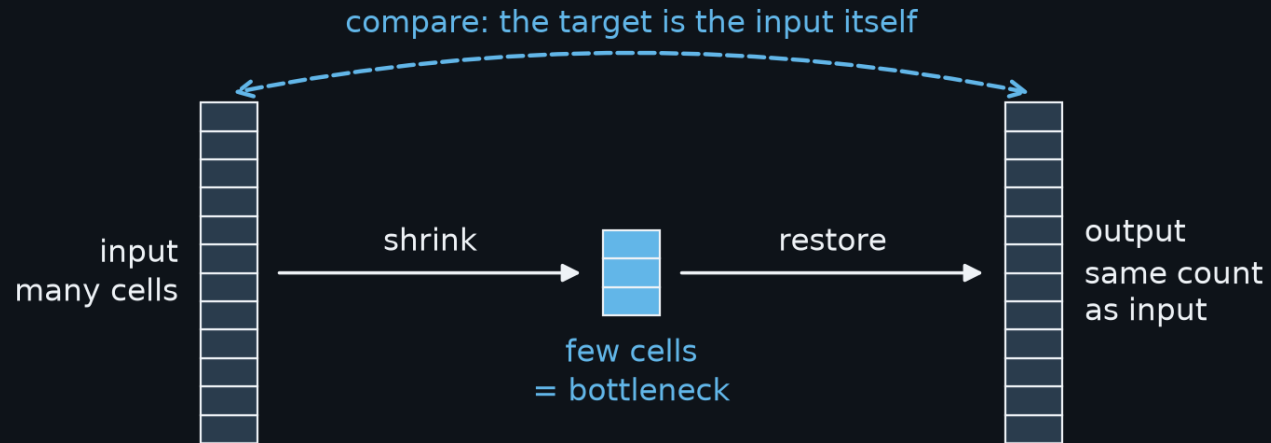
질문 — 가장 쉬운 풀이

- 정답이 입력 자신이면, 차이를 줄이는 가장 쉬운 방법은
- 그렇게 풀면 모델은 이미지에 대해 무엇을 알게 될까

그대로 베끼기

- 받은 것을 그대로 내보내면 차이는 0
- 이미 가진 이미지를 다시 돌려받을 뿐
- 문서든 숫자든 몰라도 되는 풀이 · 손상된 문서를 넣으면 손상된 그대로 나옴
- 생각으로 따진 것 · 실제 모델이 이렇게 되는지는 아직 보지 않음

가운데를 좁히기



- 정답은 여전히 입력 자신
- 가운데에 다 담을 자리 없음 → 무엇을 남길지 골라야 함
- 이렇게 좁힌 자리: 병목

질문 — 통로에 무엇을 남길까

- 아까 본 손상된 문서를 몇 칸짜리 통로로 보냈다가 되돌린다면
- 글자 모양 · 줄의 위치 · 얼룩 · 흠어진 점 — 무엇부터 남길까
- 이유 하나와 함께

누가 고르나

- "글자를 남겨라"라고 모델에게 알려 준 적 없음
- 주는 것은 두 가지: 정답은 입력 자신 · 가운데는 좁음
- 손상된 문서를 넣으면 정답도 손상된 문서 · 얼룩도 되돌릴 대상
- 무엇이 실제로 남는지는 아직 모름 · 오늘은 숫자 그림으로 원본과 나란히 놓고 확인

통로의 크기 — 남겨 둘 질문

- 더 좁히면 무엇이 달라질까
- 넓히면 다시 베끼기에 가까워질까 — 아직 보여 준 것 없음
- 통로 크기를 바꿔 보는 것은 오전 뒷부분

이름 — 오토인코더

- 입력을 가운데 표현으로 바꿨다가 다시 입력으로 되돌리도록 학습한 모델
- 오늘 첫 모델: 가운데를 좁게 뚫 · 좁은 통로는 여러 설계 중 하나
- 정답 자리에 입력 자신 · auto: '자기 자신'이라는 뜻의 말
- '자동'이라는 뜻 아님 · 무엇으로 학습할지 · 통로를 얼마나 좁힐지는 사람이 정함

확인 — 맞는 말일까

- "오토인코더는 라벨이 없으니, 학습할 때 정답도 없다"
- 맞다 · 틀리다 하나와 이유 하나

라벨 없이 배우는 쪽

	사람이 붙인 라벨	손실에서 출력과 비교하는 것
라벨을 맞히는 학습	필요	라벨
오토인코더	필요 없음	입력 자신

- 정답이 없는 것이 아니라, 정답 자리에 입력이 들어가는 것
- 데이터만으로 배움 → 머신러닝에서 배운 비지도 학습과 같은 편

오늘의 순서

순서	할 일
1	좁은 통로 모델을 하나 만들어 학습시키기
2	통로 크기 바꿔 보기
3	가운데 통로에 남은 숫자를 점으로 찍어 보기
4	합성곱으로 다시 만들기 · 줄이고 늘린 크기 맞추기
끝	1일차 정리

- 무엇이 나올지 미리 말하지 않음 · 예측 → 실행 → 확인
- 시연: 화면으로 코드와 결과를 같이 봄
- 실습 P1 ~ P4: Colab 에서 직접 씀

오늘 연습할 데이터

- 문서 대신 손글씨 숫자 이미지 · MNIST
- 작고 단순한 이미지로 원리부터 봄
- 문서는 이미지가 크고 손상이 여러 겹 · 문서로 옮기는 것은 그 뒤

실습 준비 확인

- 첫 실습 파일 `PRACTICE_P1.ipynb` 를 Colab 에서 엮
- 맨 위 '준비' 셀 실행 → 맨 아래 `준비 끝` 줄 확인
- 실습마다 파일이 따로 있음 · 뒤 실습은 그때 새 파일을 열고 맨 위 준비 셀부터 · 빈칸은 수업 중에 풀

다음 질문 — 좁은 통로를 코드로

- 줄이는 쪽 · 되돌리는 쪽을 무엇으로 쌓을까
- 새 층이 필요할까, 지금까지 본 층으로 될까

좁은 통로로 되돌리기

손글씨 숫자로 직접 만들기

숫자로 옮기면

- 28×28 숫자 그림 한 장 = 밝기 숫자 784개
- 가운데에 남길 수 있는 숫자는 32개
- 무엇을 남길지는 학습이 정함

0. 준비 — 데이터

```
tr = datasets.MNIST("data", train=True, download=True)
te = datasets.MNIST("data", train=False, download=True)
Xtr = tr.data.float().div(255).unsqueeze(1)    # 학습 60,000장 · 값 0~1
Xte = te.data.float().div(255).unsqueeze(1)    # 평가 10,000장
yte = te.targets                               # 평가 이미지의 숫자 라벨 — 학습에는 쓰지 않음
```

- MNIST: 0~9 손글씨 숫자 흑백 그림
- `div(255)` : 밝기 0~255 → 0~1
- `unsqueeze(1)` : '그림 한 장이 몇 층' 자리 추가 · 흑백 = 1층

준비 셀 실행

- 데이터 불러오기 · 함수 세 개 만들기
- 출력: 학습 · 평가 데이터 모양 한 줄

출력 읽기

0 1 2 3 ← position
(N, C, H, W)

N: how many images

C: layers per image (gray 1, color 3)

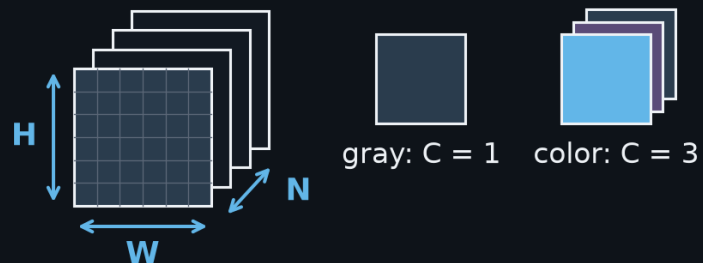
H: rows

W: columns

PyTorch counts positions from 0
(dimension 3 = W, the columns)

$(1, 1, 28, 28) = 1 \text{ image} \cdot 1 \text{ layer} \cdot 28 \text{ rows} \cdot 28 \text{ columns}$

$(128, 1, 28, 28) = 128 \text{ images}$



- 학습 $(60000, 1, 28, 28)$ · 평가 $(10000, 1, 28, 28)$
- 자리 번호는 0부터: 0 장 수 N · 1 층(채널) C · 2 세로 H · 3 가로 W
- 한 장 = $28 \times 28 = 784$ 개 값 · 값은 0~1

함수 ① run — 결과만 뽑기

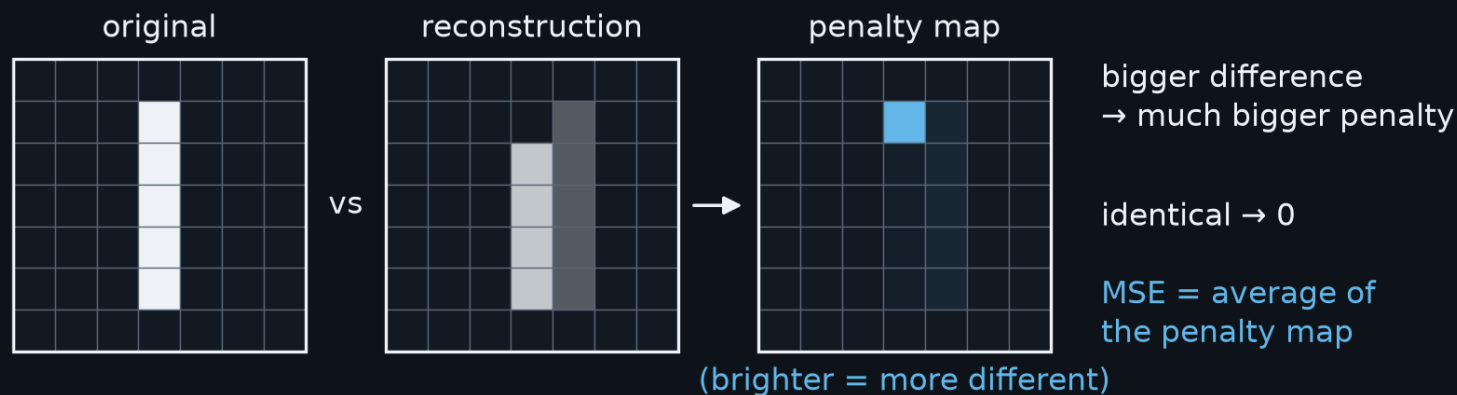
```
@torch.no_grad()
def run(model, X):                                # 한꺼번에 넣으면 메모리가 커서 2,000장씩
    model.eval()
    return torch.cat([model(X[i:i + 2000]) for i in range(0, len(X), 2000)])
```

- 약속: 모델 + 이미지 묶음 넣음 → 되돌린 묶음 나옴 · 안쪽은 외우지 않아도 됨
- `@torch.no_grad()` : 기울기 계산 끄 · 결과만 봄
- `model.eval()` : 결과만 볼 때 켜는 스위치
- 2,000장씩 넣고 `torch.cat` 으로 이어 붙임

함수 ② mse — 원본과 얼마나 다른지

```
def mse(A):  
    return F.mse_loss(A, Xte).item()
```

평가 10,000장 전체를 원본과 비교



- 많이 다른 칸일수록 벌점이 훨씬 커짐 · 똑같으면 0 · 평가 그림 전체의 평균

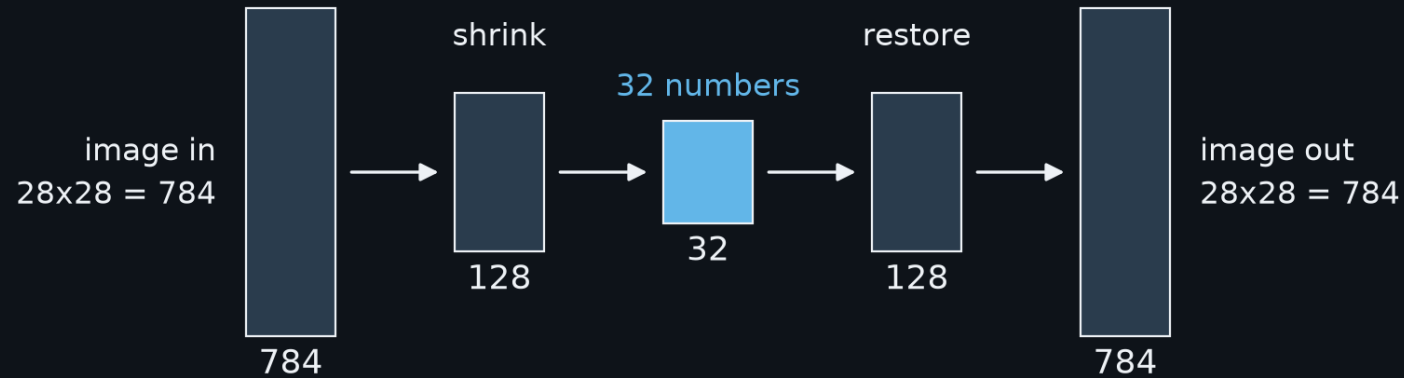
함수 ③ show — 나란히 그리기

```
SHOW = [5459, 9898, 926, 1150, 5691, 2400, 7792, 950] # 평가 이미지 중 숫자 0~7 한 장씩

def show(rows):
    fig, axes = plt.subplots(len(rows), 8, figsize=(8, 1.15 * len(rows)))
    for r, (name, X) in enumerate(rows):
        for c in range(8):
            axes[r, c].imshow(X[SHOW[c], 0], cmap="gray", vmin=0, vmax=1)
            axes[r, c].axis("off")
        axes[r, 0].set_title(name, loc="left", fontsize=10)
    plt.tight_layout()
    plt.show()
```

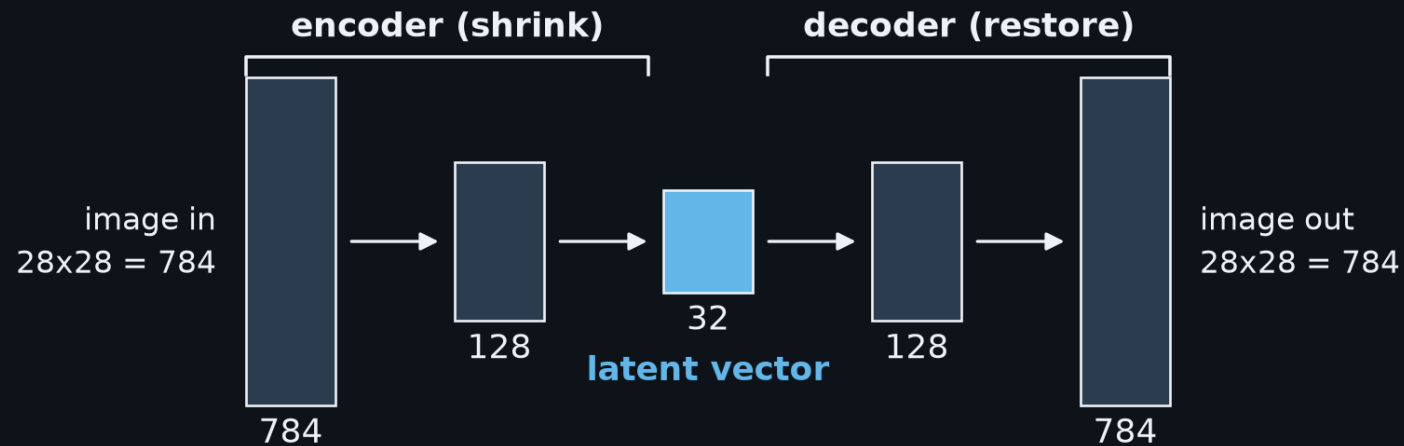
- `rows` : (줄 이름, 이미지 묶음) 목록 · 한 줄에 여덟 장
- `vmin=0, vmax=1` : 모든 칸의 밝기 기준을 0~1 로 고정

만들 모델의 모양



- 넣는 것: 숫자 그림 한 장 · 784개 값
- 내놓는 것: 같은 크기의 그림 · 784개 값
- 가운데 32칸: 784개를 한 칸씩 담을 자리가 없음

이름 붙이기

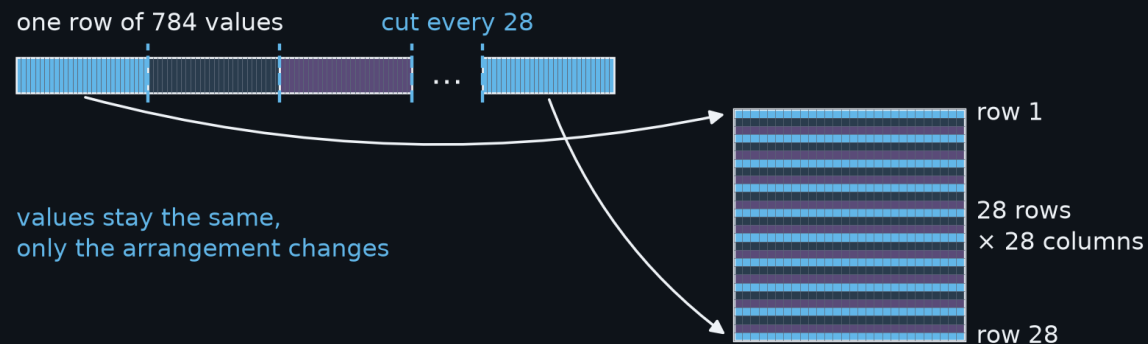


- 줄이는 쪽 = 인코더 · 되돌리는 쪽 = 디코더
- 가운데 통로에 남은 숫자 묶음 = 잠재 벡터
- 잠재: 겉으로 드러나지 않고 안에 숨어 있음

조립 셀 — 뼈대

```
class AutoEncoder(nn.Module):  
    def __init__(self, latent_dim=32):  
        super().__init__()  
        # ◆ 인코더 — 여기에 칠 것  
        # ◆ 디코더 — 여기에 칠 것  
    def forward(self, x):  
        return self.decoder(self.encoder(x)).reshape(-1, 1, 28, 28)
```

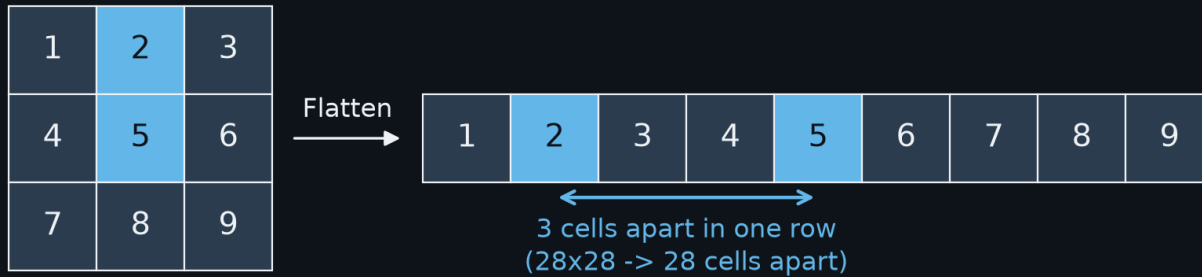
- `latent_dim=32` : 잠재 벡터 크기 · 기본값 32
- `forward` : 인코더 → 디코더 → 28×28 모양으로 접기



인코더의 첫 부품 — Flatten

	모양
들어오는 그림	(장 수, 1, 28, 28)
<code>nn.Flatten()</code> 뒤	(장 수, 784)

3x3 image: 2 and 5 are neighbors (above / below)

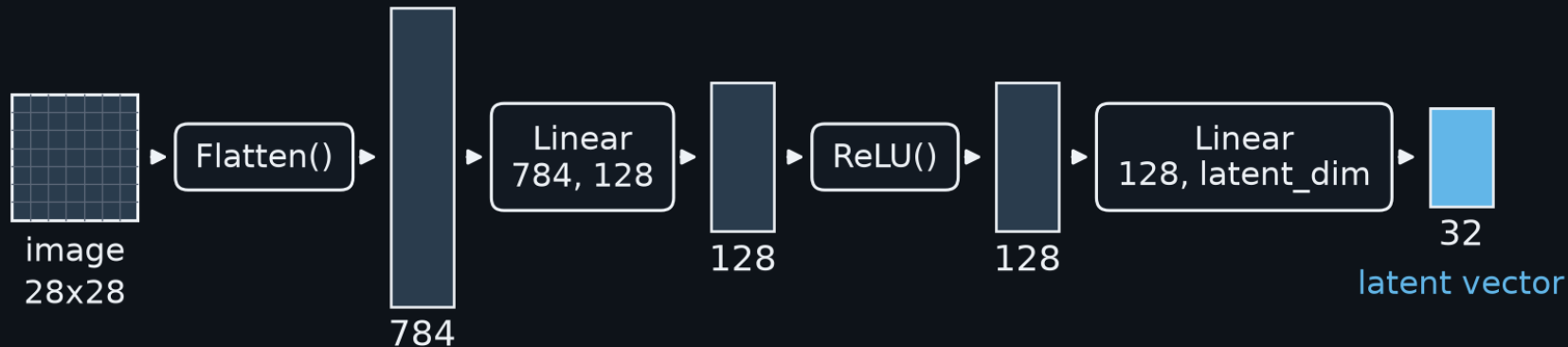


이번에 칠 줄 — 인코더

```
self.encoder = nn.Sequential(nn.Flatten(), nn.Linear(784, 128), nn.ReLU(), nn.Linear(128, latent_dim))
```

- `nn.Sequential` : 부품을 적은 순서대로 이어 붙임

`nn.Sequential`: parts run in the order written →



디코더 첫 부품은?

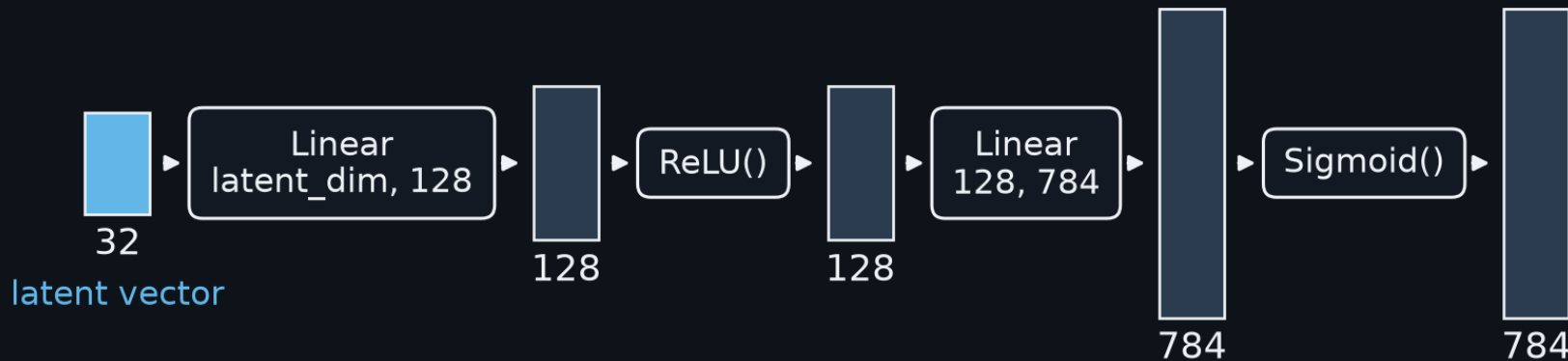
- 디코더 첫 `nn.Linear` 는 값을 몇 개 받아야 할까
- 인코더가 마지막에 내놓은 것을 떠올려 봄

이번에 칠 줄 — 디코더

```
self.decoder = nn.Sequential(nn.Linear(latent_dim, 128), nn.ReLU(), nn.Linear(128, 784), nn.Sigmoid())
```

- 인코더를 거꾸로 · 마지막에 `nn.Sigmoid()`

decoder: the encoder in reverse →

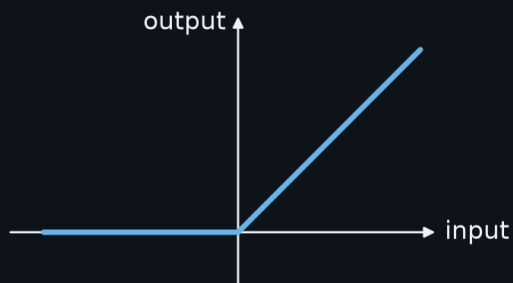


왜 마지막에 Sigmoid?

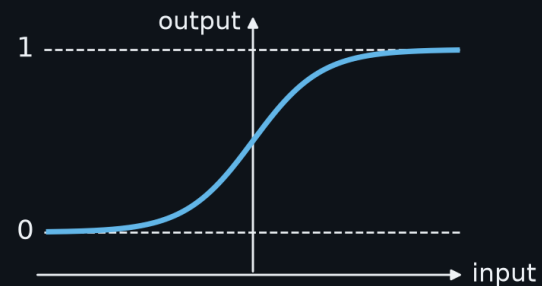
- 원본 픽셀은 준비 셀에서 0~1로 맞춰 줌
- 디코더 끝에 이 함수를 둔 이유 하나

값의 범위 — 마지막 Sigmoid

ReLU: negative $\rightarrow$ 0, positive $\rightarrow$ same



Sigmoid: any value $\rightarrow$ between 0 and 1



- `nn.Linear` 출력: 범위 제한 없음 · 음수도 1 넘는 값도 가능
- `nn.Sigmoid()` : 픽셀마다 0~1 사이로 눌러 넣음 $\rightarrow$ 원본 픽셀(0~1)과 같은 범위

셀 아래쪽 — 모양을 세 번 찍음

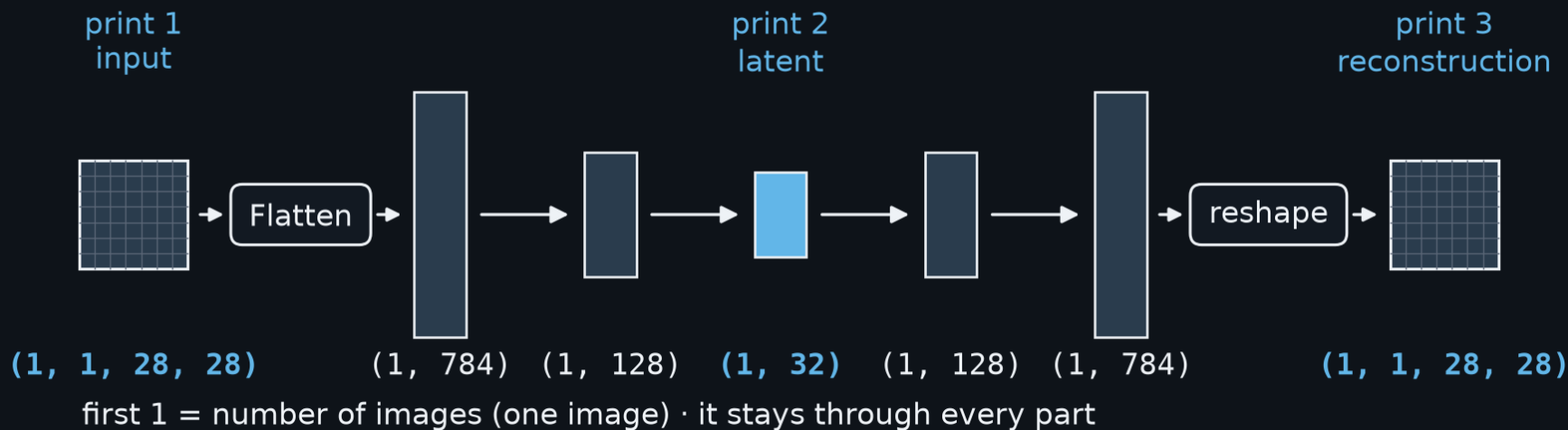
```
torch.manual_seed(0)
ae0 = AutoEncoder() # 학습 전 모델 — 뒤에서 비교용으로 남겨 둠
x = Xte[:1]
print("입력 모양", tuple(x.shape))
print("잠재 벡터 모양", tuple(ae0.encoder(x).shape))
print("복원 모양", tuple(ae0(x).shape))
```

- 입력 · 인코더만 지난 잠재 벡터 · 모델 전체를 지난 복원의 모양

조립 셀 실행

- 인코더 · 디코더 두 줄을 친 뒤 실행
- 출력 네 줄: 모양 세 줄 · 마지막 한 줄

모양 세 줄



- 파란 꼬리표 세 개 = print 로 찍은 세 줄
- 맨 앞 1 = 장 수 자리(한 장) · 층을 지나도 그대로
- 입력과 복원의 모양이 같음 · 픽셀끼리 비교 가능

마지막 줄 — 학습 전 복원 MSE

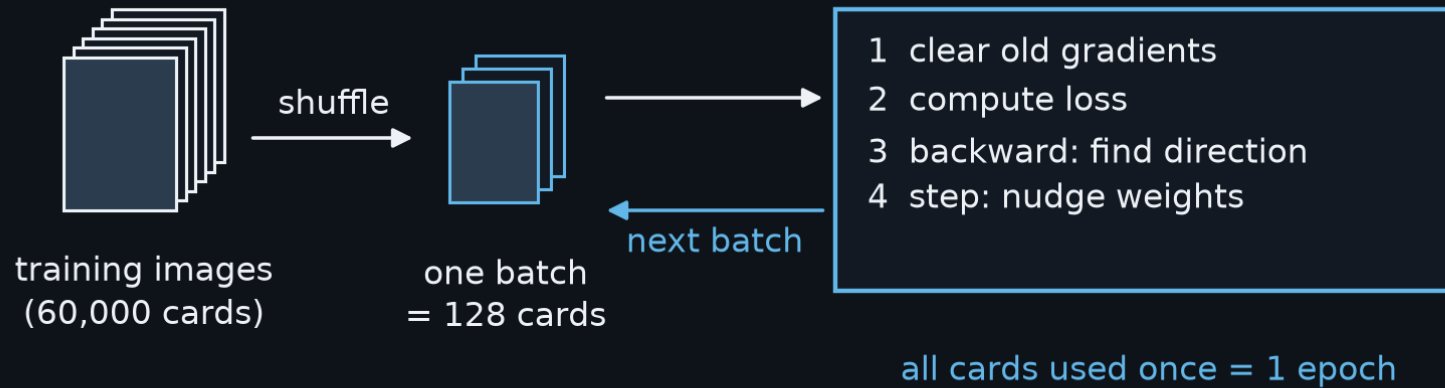
- 학습 전 모델 `ae0` 으로 평가 이미지 10,000장을 되돌림
- `mse` 로 원본과 비교: 0.232
- 복원 오차: 되돌린 그림이 원본과 얼마나 다른지를 잴 값

예측 ①

784개를 32개로 줄였다가 되돌리면, 학습 뒤 복원은 원래 숫자로 알아볼 수 있을까?

- ① 원래 숫자로 알아볼 수 있음
- ② 뭉개져서 알아보기 어려움
- ③ 그 밖
- 번호 하나 · 이유 하나

학습이 도는 모양 — 카드 더미



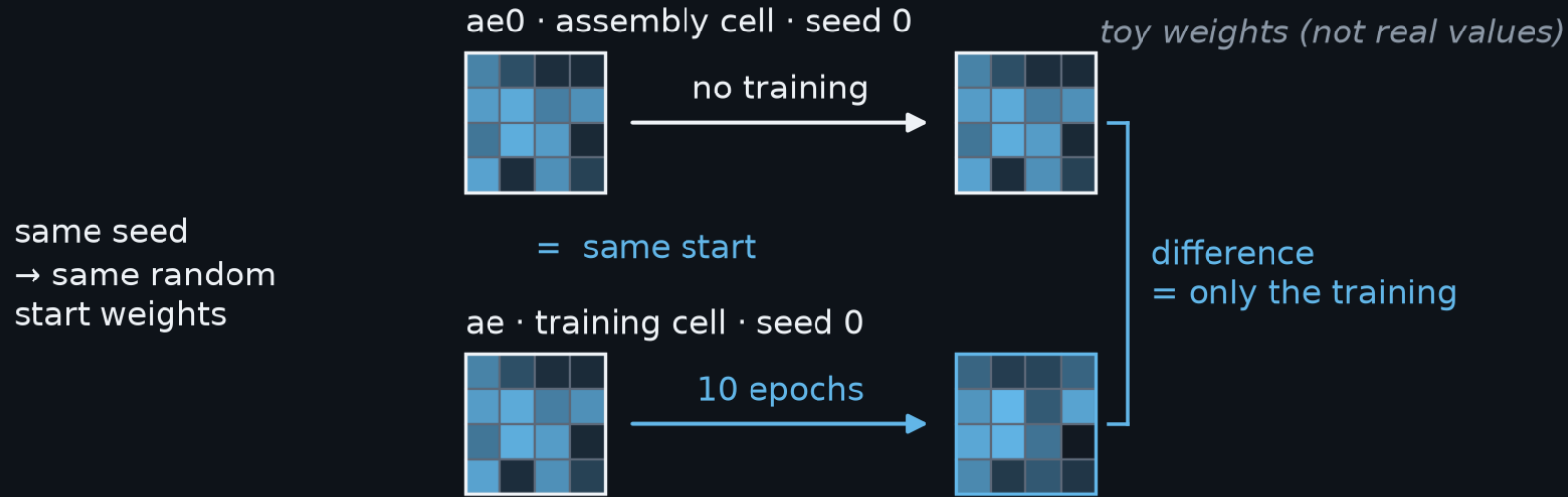
- 옵티마이저: 가중치를 고치는 담당
- 묶음(배치): 한 번 고칠 때 보는 그림 128장
- 에폭: 학습 그림 전체를 한 바퀴 다 봄

2. 학습 — 루프 뼈대

```
torch.manual_seed(0)
ae = AutoEncoder()
opt = torch.optim.Adam(ae.parameters(), lr=1e-3)
EPOCHS = 10
for epoch in range(EPOCHS):
    ae.train()
    t = time.time()
    perm = torch.randperm(len(Xtr))
    for i in range(0, len(Xtr), 128):
        x = Xtr[perm[i:i + 128]]
        opt.zero_grad()
        # ◆ 정답 자리에도 입력 x — 여기에 칠 것
        loss.backward()
        opt.step()
```

- CNN · RNN 에서 본 학습 루프와 같은 모양
- 새로 칠 곳: 손실 한 줄

시드 0 — 같은 출발점



- **ae0** : 조립 셀 · 학습 안 함 · **ae** : 학습 셀 · 10 에폭
- 같은 시드 → 시작 가중치가 같음 · 두 모델의 차이 = 학습이 만든 것

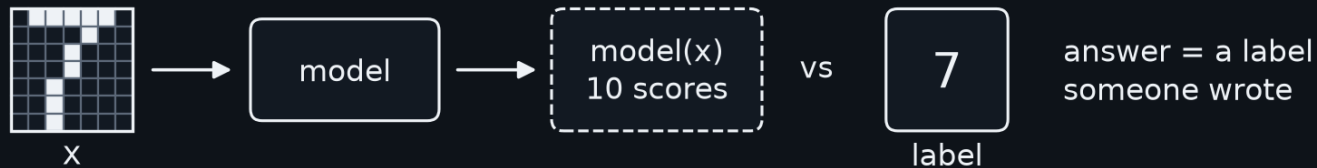
이번에 칠 줄 — 손실

```
loss = F.mse_loss(ae(x), x)
```

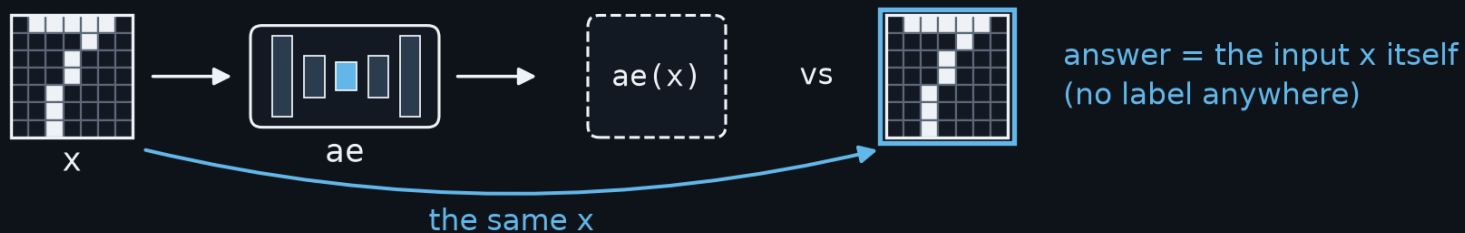
- `ae(x)` : 모델이 되돌린 그림
- 뒤의 `x` : 정답 자리 · 입력 그대로

이 모델이 푸는 문제

classifier (CNN course)

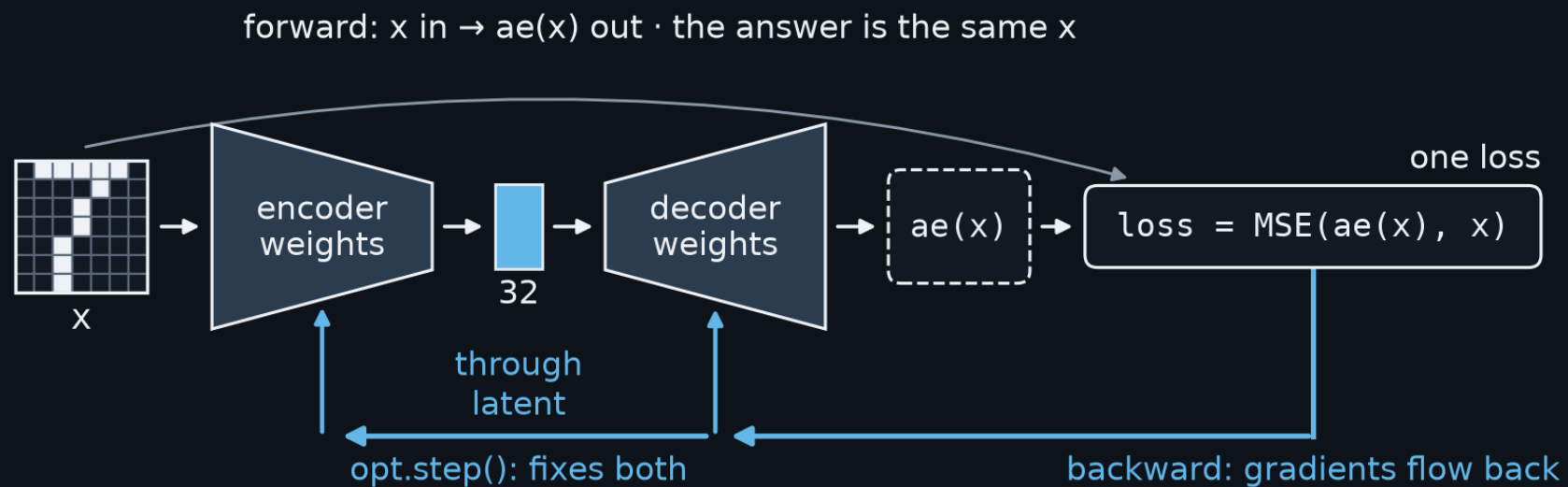


autoencoder (today)



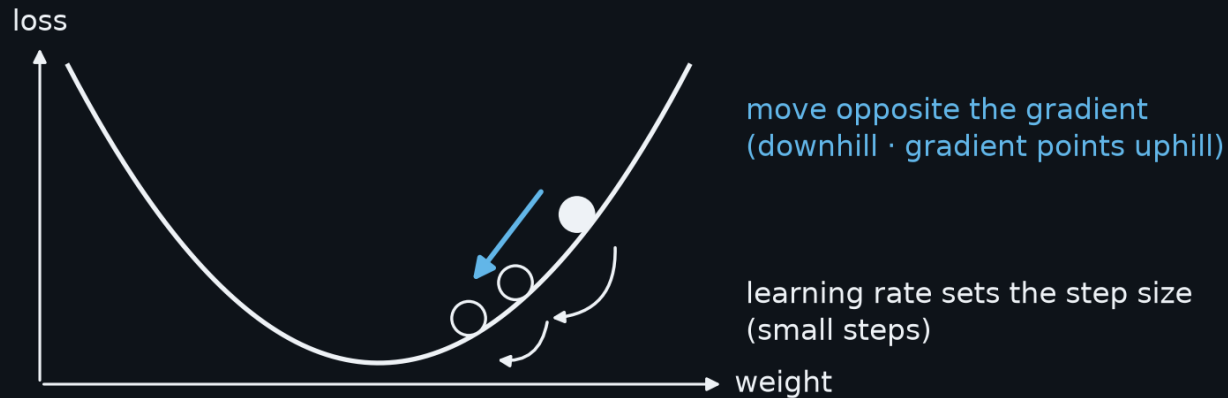
- 라벨(몇 번 숫자인지): 학습 루프 어디에도 없음
- 손실 = 복원과 입력의 MSE · 재구성 오차
- 라벨 없이 데이터만으로 배움 · 머신러닝에서 배운 비지도 학습

손실은 하나 · 고쳐지는 곳은 둘



- 먼저 `opt.zero_grad()` : 앞 묶음에서 쌓인 기울기를 0 으로 · 안 비우면 계속 더해짐
- `Adam(ae.parameters())` : 인코더 · 디코더 가중치 전부

기울기 · 학습률 — 비탈 내려가기



- 기울기: 이 자리의 비탈 방향 · 가파름 → 그 **반대쪽**(내리막)으로 옮김
- 학습률: 보폭의 크기를 정하는 값 · 작게 둬

이 셀에서 정한 값

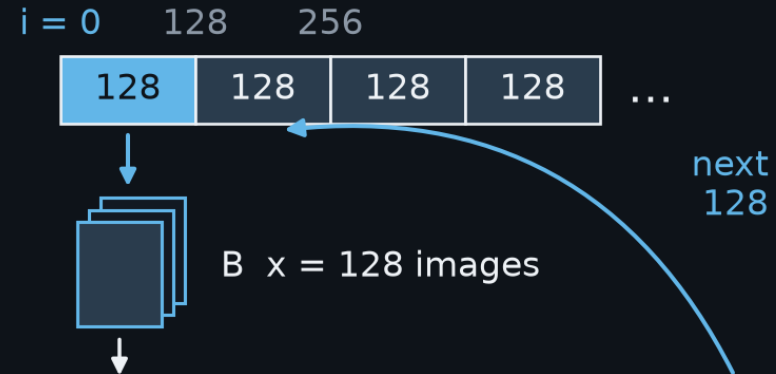
고른 것	값
손실	MSE · 되돌린 픽셀 값을 원본 값에 가깝게
옵티마이저	Adam(가중치를 고치는 담당) · 학습률 1e-3(한 번 고치는 보폭)
묶음 · 에폭	128장 · 10 에폭

- 다른 값과 비교해서 고른 것은 아님

학습 돌리기

```
D | for epoch in range(EPOCHS):  
A |     perm = torch.randperm(len(Xtr))  
B |     for i in range(0, len(Xtr), 128):  
        x = Xtr[perm[i:i + 128]]  
        opt.zero_grad()  
        loss = F.mse_loss(ae(x), x)  
C |        loss.backward()  
        opt.step()
```

A shuffled order of 60,000 image numbers

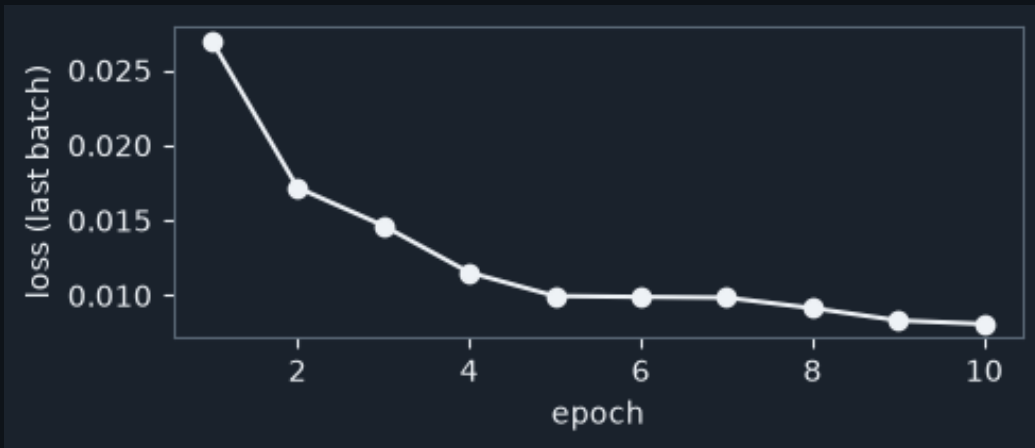


C zero_grad → loss → backward → step

D all used = 1 epoch → shuffle again

- '2.' 셀 · 10 에폭 · 128장씩

출력 — 에폭마다 한 줄



- 첫 에폭 0.6초 · 이 컴퓨터 CPU 기준
- loss 선: 에폭마다 마지막 한 묶음에서 잦 값 · 내려가는 모양만 봄

그림을 띄우기 전에

- 학습 전 모델 `ae0` 은 어떤 그림을 내놓을까

3. 학습 전과 후

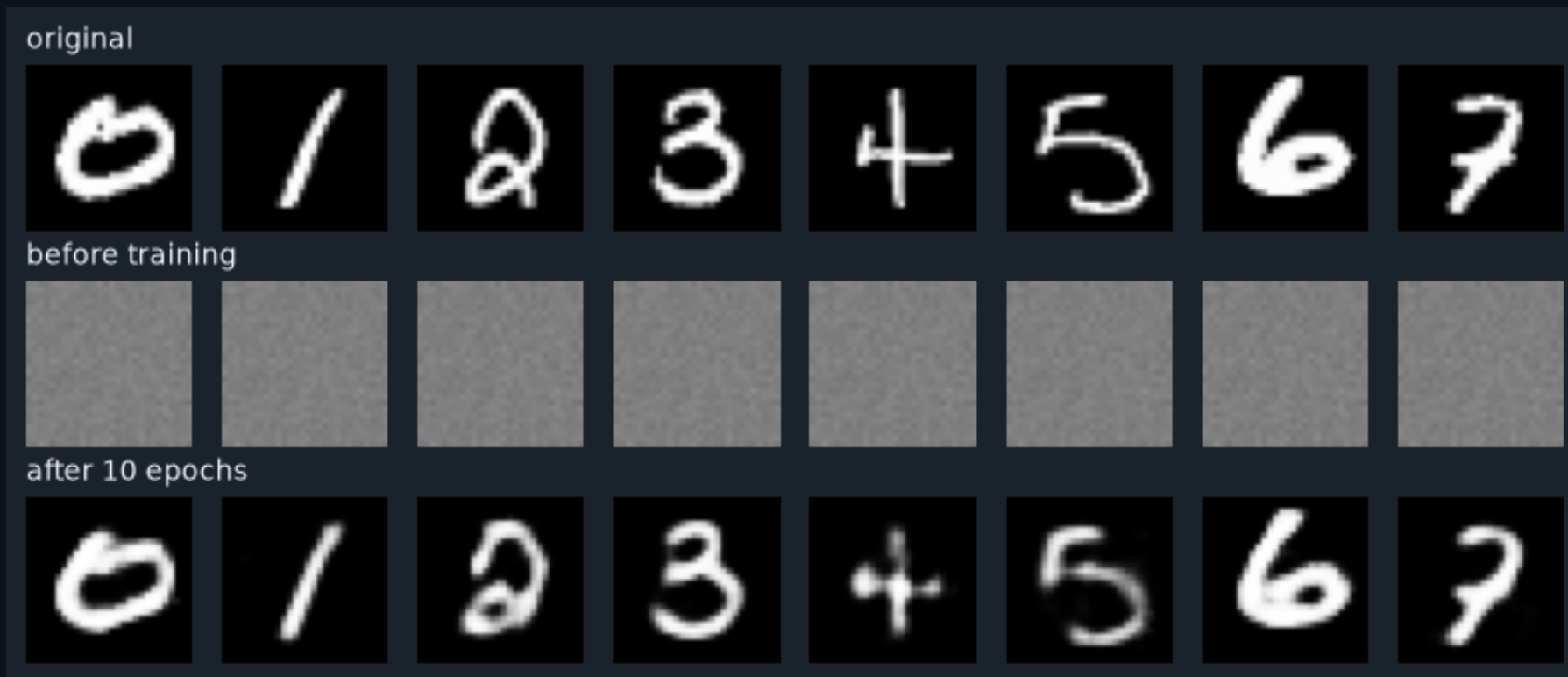
- 세 줄: original · before training · after 10 epochs
- 열: 평가 이미지 숫자 0~7 한 장씩 · 줄마다 같은 이미지
- 읽는 법: 한 열씩 원본과 위아래로 맞대기

가운데 줄 — before training



- 여덟 칸 모두 자글자글한 회색 무늬
- 숫자 모양 없음 · 칸끼리 구별 안 됨

아래 줄 — after 10 epochs · 잘 돌아온 것



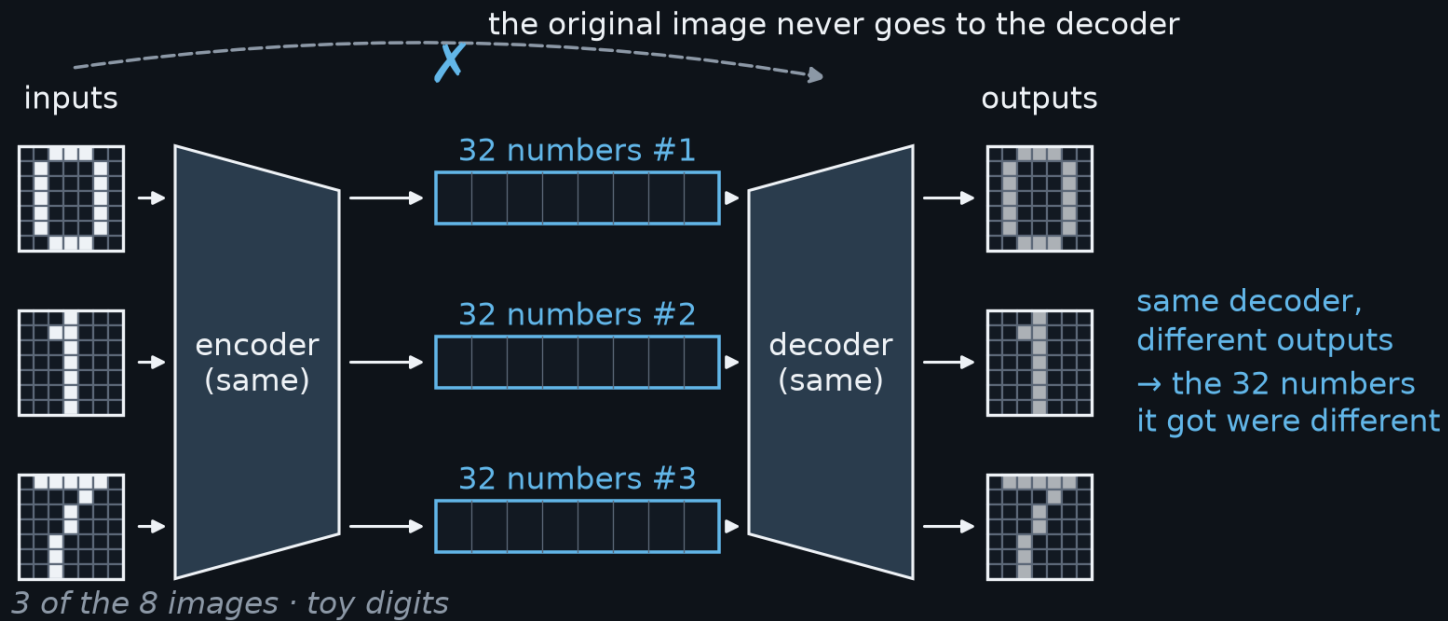
- 0 · 1 · 3 · 6: 원본과 모양이 거의 같음
- 0: 획이 조금 두꺼워지고 왼쪽 위에 겹친 작은 고리는 거의 사라짐
- 모든 열: 획 가장자리가 원본보다 부드럽게 번짐

아래 줄 — 흐리거나 달라진 것



- 4: 획이 흐리고 끊김 · 가로획 왼쪽 끝과 가운데에 점처럼 뭉친 곳
- 5: 아래 곡선이 흐리고 둘레에 회색 번짐
- 2: 아래 고리 부분 획이 뭉쳐 두꺼워짐
- 7: 윗가로획 왼쪽 끝이 둥글게 말림 · 가운데 짧은 가로획은 작은 돌기만 남음

디코더가 받은 것은 숫자 32개뿐



- 같은 디코더에서 서로 다른 모양이 나옴 → 받은 32개가 서로 다름
- 32개에 무엇이 담겼는지는 이 그림으로는 모름

실행 — '3-1. 숫자로 확인'

- 학습 전 `ae0` · 학습 뒤 `ae` 의 복원 MSE 를 한 줄씩 찍음

숫자로 확인 — 평가 이미지 10,000장

	복원 MSE
학습 전 <code>ae0</code>	0.232
10 에폭 학습 뒤 <code>ae</code>	0.008

- 같은 평가 이미지 · 같은 계산 · 0 에 가까울수록 원본과 같음 · 학습 뒤가 훨씬 작음
- 에폭마다 찍힌 loss: 한 묶음에서ぜん 값 · 여기 복원 MSE: 평가 1만 장 전체

예측 ① 확인

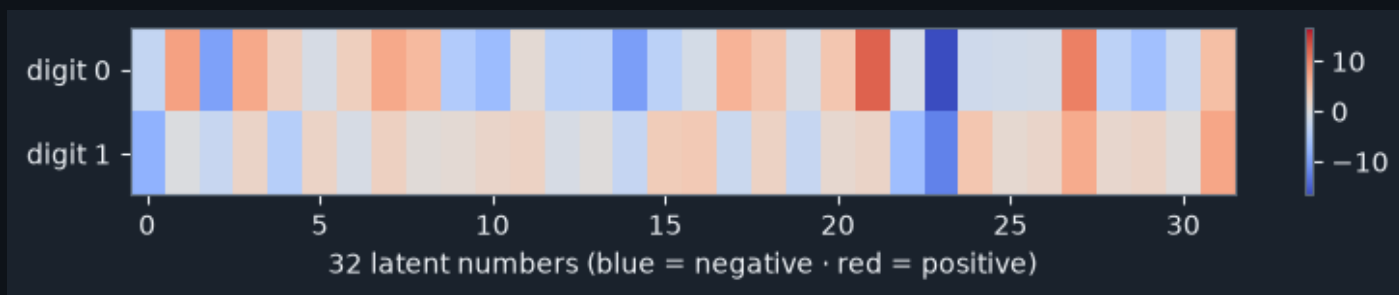
	학습 전	10 에폭 뒤
그림	회색 무늬 · 숫자 없음	여덟 장 중 대부분 원래 숫자로 읽힘
복원 MSE	0.232	0.008

- 이번 실행: 대부분 알아볼 수 있음 · 원본보다 흐림 · 4 는 읽기 어려움

실행 — '3-2. 잠재 벡터 들여다보기'

- 숫자 0 · 1 그림 두 장을 인코더에만 넣어 봄
- 출력: 모양 한 줄 · 앞 8개 숫자 두 줄 · 32개를 색으로 칠한 띠 두 줄

잠재 벡터 — 숫자 32개를 색 띠로



- 숫자는 읽지 않음 · 색 띠 두 줄의 무늬가 다름
- 음수(파랑) · 양수(빨강)가 섞인 평범한 숫자 묶음
- 숫자 하나하나의 뜻은 정해 준 적 없음

해보셈

본 것과 추측 나누기 · 첫 실습 P1

본 것 · 추측 · 확인 못 한 것

구분	이번 시간에
본 것	여덟 장 중 4 는 읽기 어렵고 5 는 흐림 · 복원 MSE 가 학습 전보다 작아짐
추측	왜 4 가 유독 달라졌나 — 여러분의 생각
확인 못 한 것	그 추측이 맞는지 · 1만 장 전체에서도 같은 모습인지

- 추측을 확인하려면 그것을 겨냥한 실험이 따로 필요함

오토인코더 · AutoEncoder

부분	하는 일
인코더	784개 → 잠재 벡터 32개로 줄임
디코더	32개 → 784개로 되돌림
학습	정답 자리에 입력 자신 · 재구성 오차를 줄임

- auto: 자기 자신(아침에 본 뜻)
- AE: AutoEncoder 를 줄인 이름

이번 시간에 한 것

- 입력과 정답 적기: 입력 $\times$ · 정답 $\times$
- 크기를 한 줄씩 적기: $784 \rightarrow 128 \rightarrow 32 \rightarrow 128 \rightarrow 784$
- 학습 전과 후를 숫자와 그림으로 나란히
- 그림은 한 열씩 원본과 맞대어 읽기 · 본 것과 추측 나누기

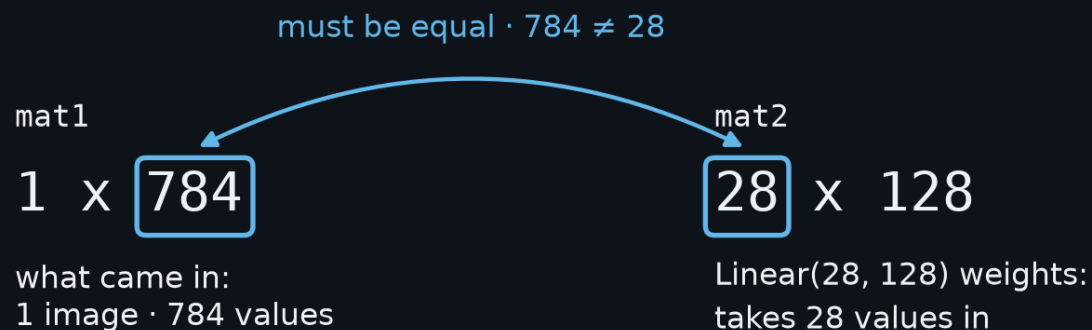
에러가 나면 — 맨 아래 줄부터

[안내] Linear 에 들어온 값 개수와 Linear 첫 숫자가 다름

RuntimeError Traceback (most recent call last)

... 화살표 줄 . torch 안쪽 칸 여러 개 ...

RuntimeError: mat1 and mat2 shapes cannot be multiplied (1x784 and 28x128)



- 먼저 [안내] 문장 · 무엇이 틀렸는지는 맨 아래 줄
- 첫 줄은 에러 종류 제목 · 중간 칸(화살표 · torch 안쪽)은 처음엔 건너뛰
- 노트북 맨 위에 자주 나오는 에러 표

실습 P1 — 크기 먼저 적고 오토인코더 채우기

PRACTICE_P1.ipynb

1. 맨 위 준비 셀 실행 · 잠재 크기를 하나 정함(예: 16)
2. 층마다 나올 모양을 표에 먼저 적음 · 더블클릭해서 고치거나 종이에
3. 인코더 · 디코더 뼈대의 빈칸: `nn.Linear` 의 숫자 · 마지막 함수 이름 · 막히면 힌트
4. 확인 셀 실행 → 적어 둔 모양과 찍힌 모양 비교

P1 확인

자리	잠재 16 일 때 모양
입력	(1, 1, 28, 28)
Flatten 뒤	(1, 784)
잠재 벡터	(1, 16)
복원	(1, 1, 28, 28)

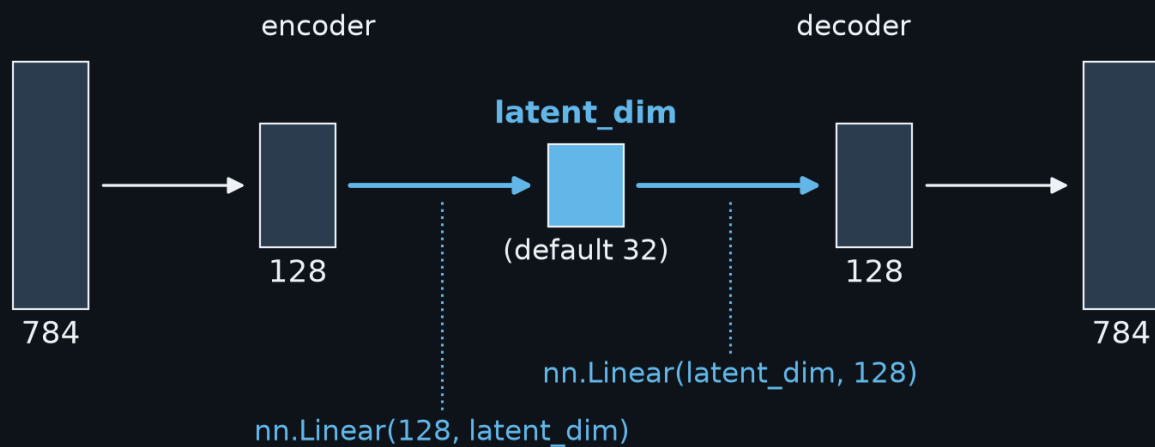
- 잠재 크기를 다르게 정했다면 잠재 벡터 줄만 다름

통로의 크기

32 대신 다른 크기라면

통로 크기를 정하는 곳

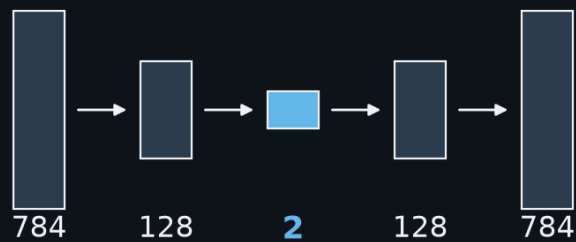
```
class AutoEncoder(nn.Module):  
    def __init__(self, latent_dim=32):
```



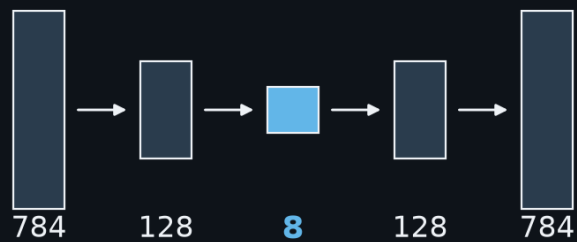
- `latent_dim` : 통로에 남길 숫자 개수 · 기본값 32 · 파란 화살표 두 자리

비교할 통로 — 잠재 $2 \cdot 8 \cdot 32 \cdot 128$

latent 2



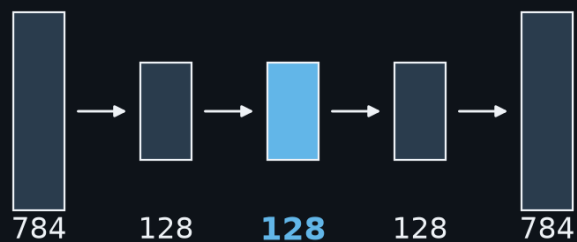
latent 8



latent 32 (model from before)



latent 128



- 바꾸는 것은 가운데 한 칸 · 앞뒤 층 · 데이터 · 10 에폭은 그대로

예측 ②

잠재를 2개까지 줄이면 복원에 무엇이 달라질까

- 784개 픽셀 → 숫자 2개 → 784개 픽셀
- 떠오르는 대로 한 가지 · 이유 하나

넓히는 쪽은

잠재를 128 로 넓히면 32 보다 나아질까

- 아주 좁은 통로를 조금 넓힐 때($2 \rightarrow 8$) · 이미 넉넉한 통로를 더 넓힐 때($32 \rightarrow 128$) — 어느 쪽 그림이 더 크게 달라질까

같은 학습을 여러 번 — `train_ae`

```
def train_ae(latent_dim, epochs=10, seed=0):  
    torch.manual_seed(seed)  
    model = AutoEncoder(latent_dim)
```

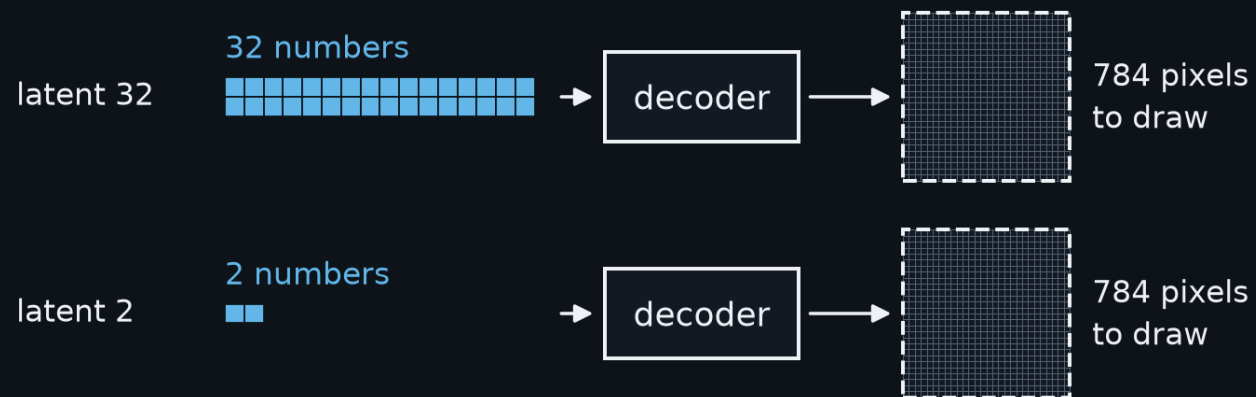
- 앞에서 본 학습 루프를 함수로 감싼 것
- 잠재 크기를 인자로 받아 새 모델을 만들고 학습
- 32 는 앞에서 본 `ae` 를 그대로 · 새로 학습은 $2 \cdot 8 \cdot 128$

실행 — 학습 세 번

- '4. 통로 크기 바꾸기' 셀
- 잠재 $2 \rightarrow 8 \rightarrow 128$ 순서로 학습
- 하나 끝날 때마다 "학습 끝" 줄

숫자 2개로 784개 그리기

the decoder gets only these numbers · the original image never goes in



- 디코더는 원본 이미지를 못 봄 · 통로를 지난 숫자만 받음

학습 끝 — 세 줄

- 잠재 2 · 6초
- 잠재 8 · 6초
- 잠재 128 · 6초

나란히 보기

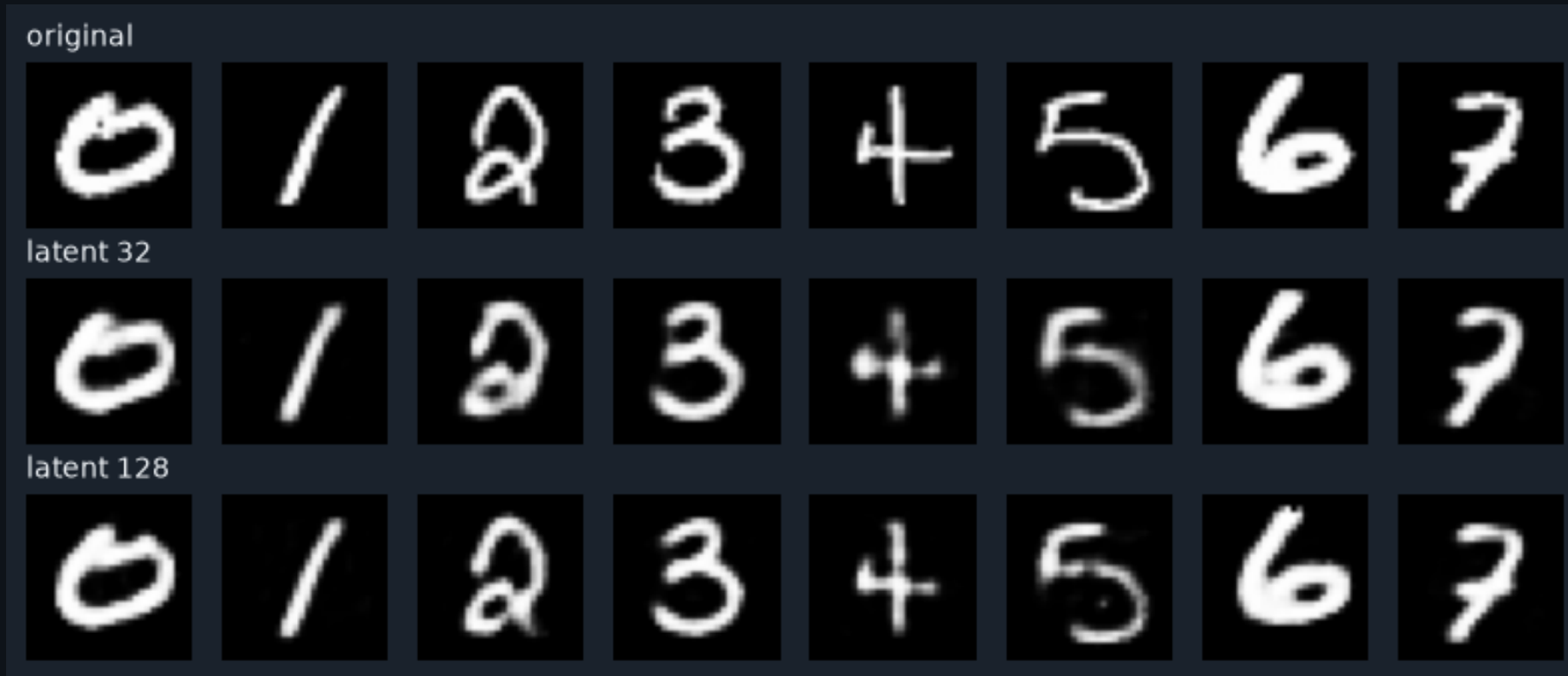
- '4-1. 나란히 보기' 셀
- 위에서부터 original · latent 2 · latent 8 · latent 32 · latent 128
- 열: 같은 평가 이미지 여덟 장 · 숫자 0~7

먼저 original 줄



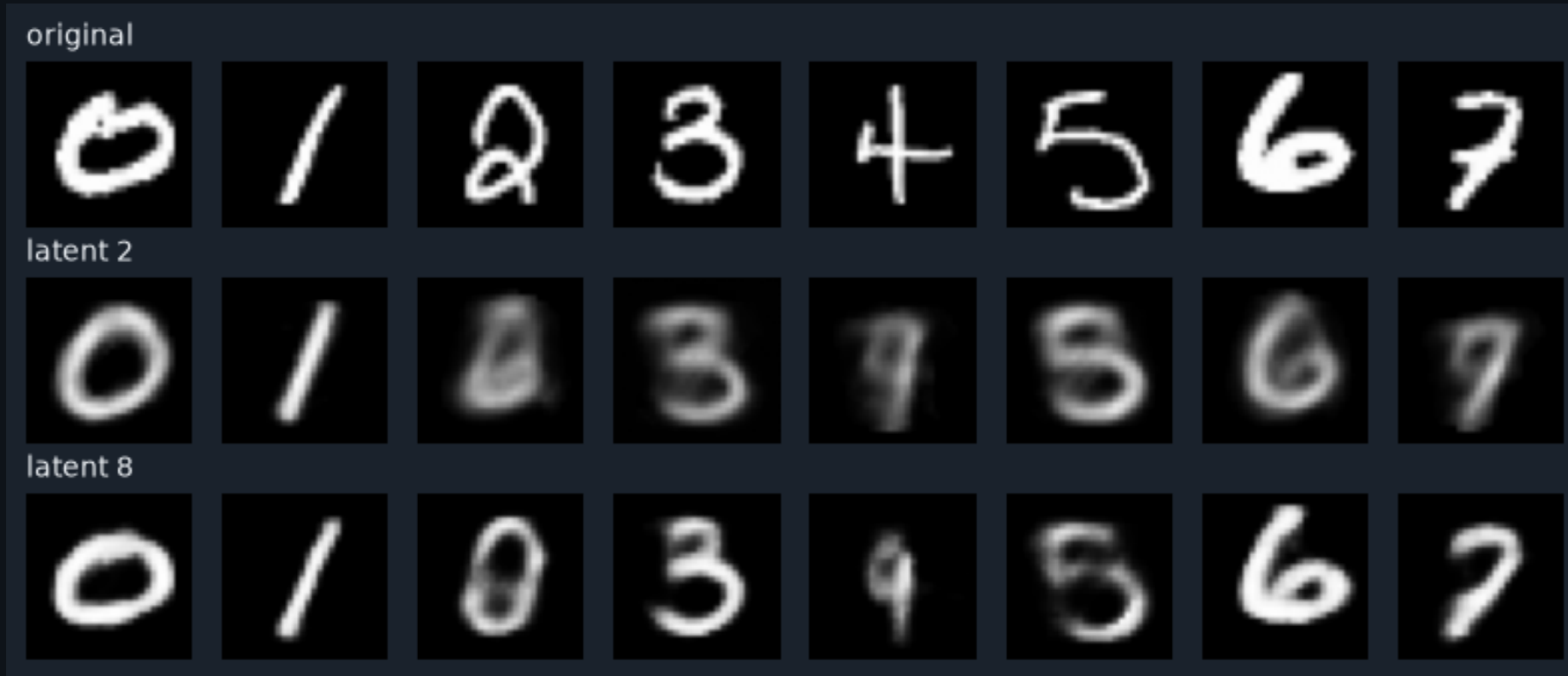
- 0: 왼쪽 위에 작은 고리 · 2: 아래쪽에 고리
- 4: 위가 열린 가는 4 · 7: 세로 획 가운데 짧은 가로줄

넓은 쪽부터 — latent 128 · latent 32



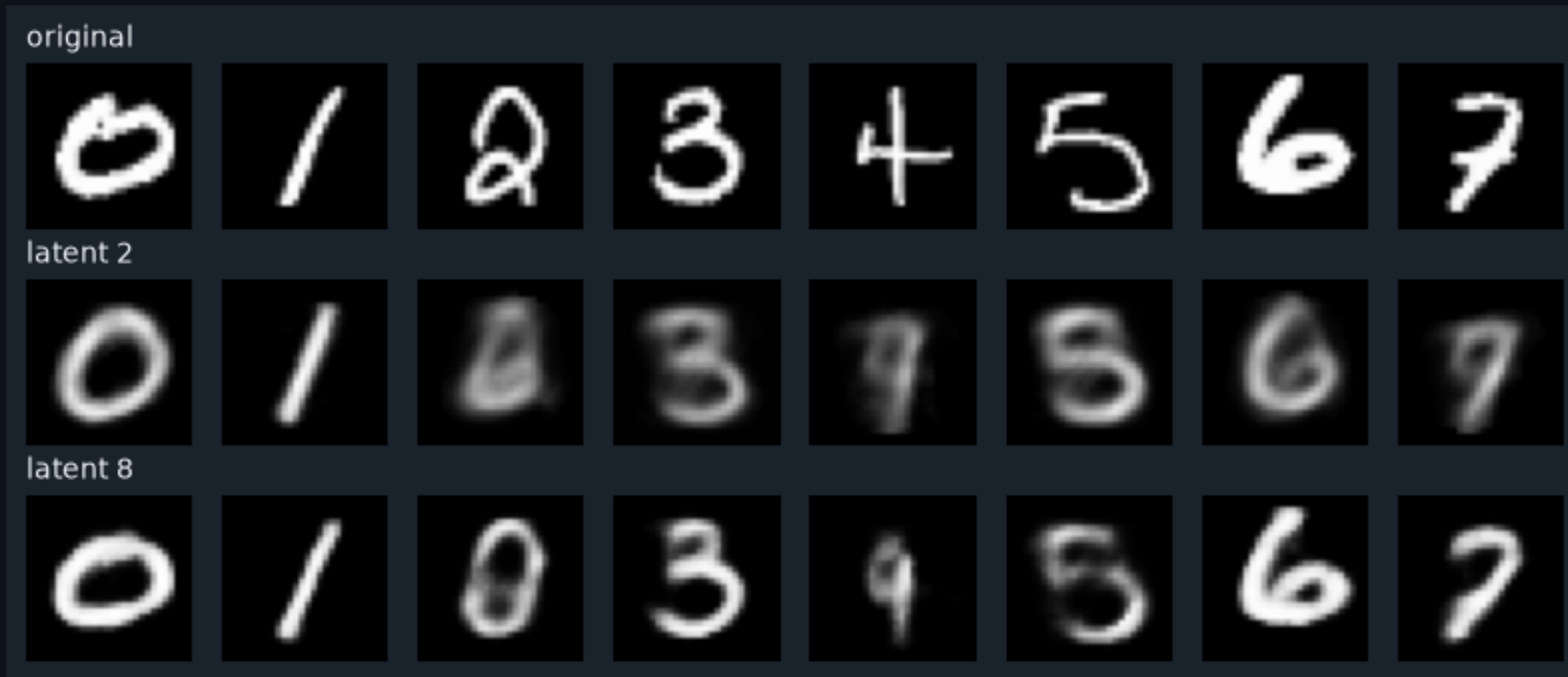
- latent 128: 여덟 칸 모두 원본과 같은 숫자로 읽힘
- latent 32: 앞에서 본 after 10 epochs 결과 같은 모델 · 4 는 획이 끊김

latent 8 줄 — 칸마다



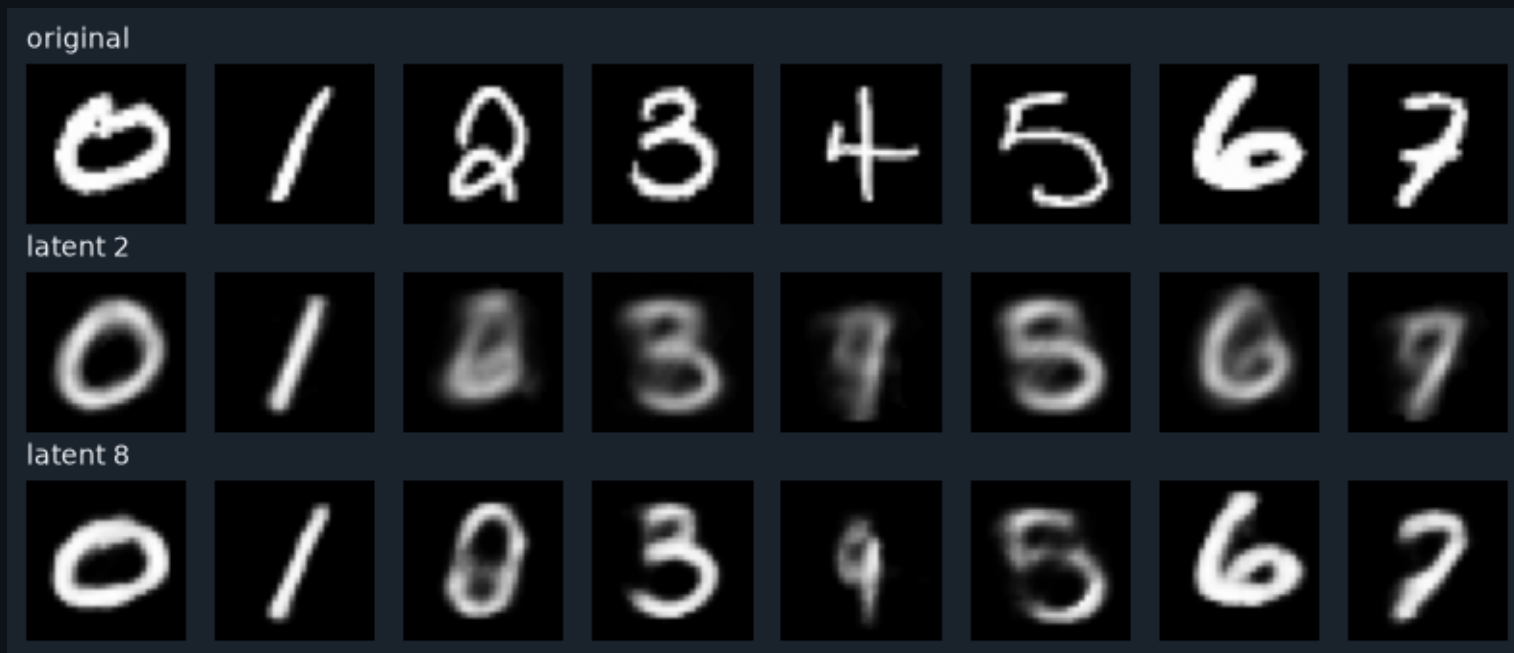
- 2 → 가운데가 막힌 모양 · 0 이나 8 에 가까움
- 4 → 작고 가늘어짐 · 9 에 가까움
- 0 · 7 → 숫자는 그대로 · 0 의 작은 고리와 7 의 가로줄이 없어짐

latent 2 줄 — 예측과 비교



- 떠올린 변화가 보이는 칸 · 다르게 나온 칸

latent 2 줄 — 칸마다



- 줄 전체가 흐림 · 테두리가 뿌옇게 번짐
- 0 · 3 · 5 · 6 → 흐리지만 원래 숫자로 읽힘 · 1은 이 줄에서 가장 또렷함
- 2 → 알아보기 어려운 덩어리 · 0이나 8 쪽 모양
- 4 → 흐린 9 처럼 보임 · 7 → 7인지 9인지 헷갈림

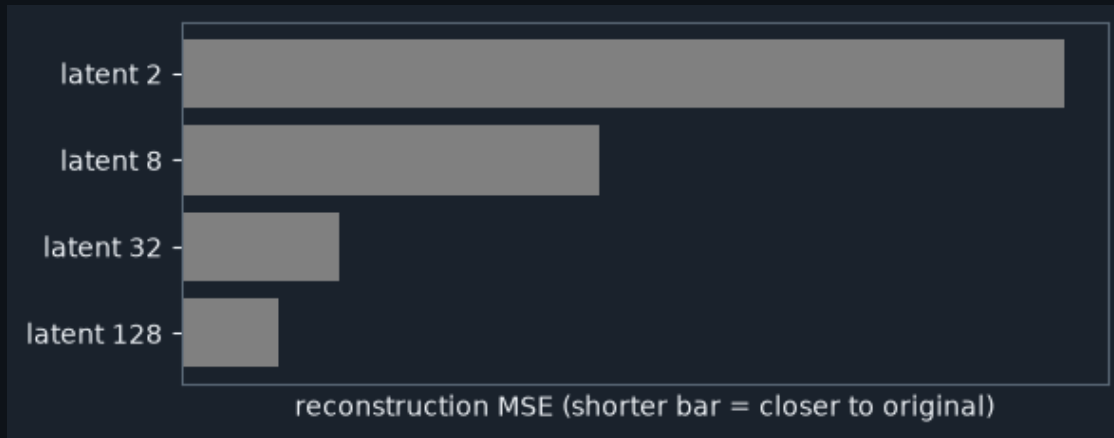
예측 ② — 이번 실행에서 달라진 것

- 흐려짐: 줄 전체
- 모양 바뀜: 2 · 4 는 다른 숫자처럼 · 7 은 헛갈림
- 모양 바뀜은 latent 8 에서도 2 · 4 에 이미 있었음

실행 — '4-2. 숫자로 확인'

- 네 모델의 복원 MSE 를 한 줄씩 찍음 · 막대 그림 하나

숫자로 확인 — 복원 MSE 네 줄

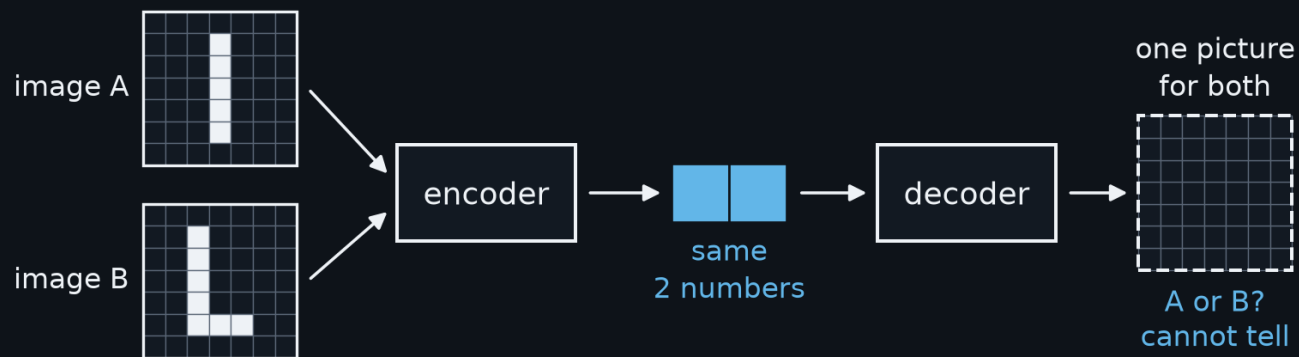


- 막대가 짧을수록 원본에 가까움(평가 10,000장 평균) · 볼 곳: 넓힐수록 짧아지나 · 가장 많이 짧아진 곳
- 값(확인용): $2 \rightarrow 0.045$ · $8 \rightarrow 0.022$ · $32 \rightarrow 0.008$ · $128 \rightarrow 0.005$

넓힐수록 덜 줄어든 까닭 하나

- MSE: 원본과 복원의 차이 · 원본과 똑같으면 0 · 0 아래로는 못 내려감
- 32 는 이미 원본에 꽤 가까움 → 더 줄어들 여지가 작음
- 2 는 원본과 멀었음 → 줄어들 여지가 큼
- 줄어든 정도가 왜 딱 이만큼인지는 이 실행으로 모름

좁힐수록 잃는 것

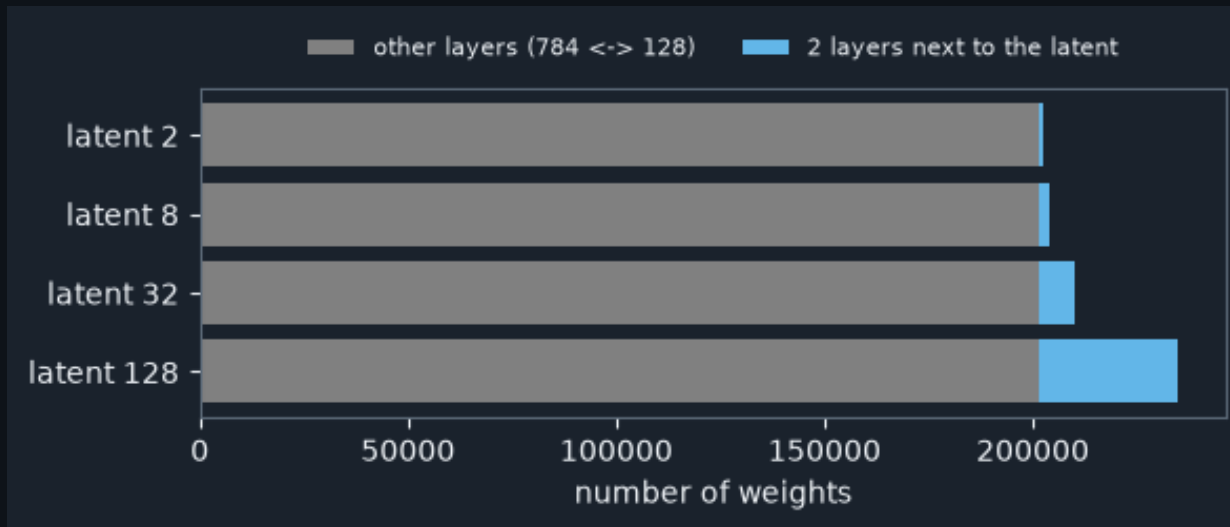


- 2 까지 좁히면 흐려질 뿐 아니라 다른 숫자 모양도 나옴
- 인코더가 두 숫자에 담지 못한 차이는 디코더가 되살릴 수 없음
- 왜 하필 그 숫자 모양이 됐는지는 이 실행으로 모름

그래도 숫자 두 개로 되돌린 것

- 앞에서 본 before training 줄: 여덟 칸 모두 회색 무늬
- latent 2: 흐리지만 0 · 1 · 3 · 5 · 6 은 알아볼 수 있음

그럼 무조건 넓게



- 이번 실행: 128 의 MSE 가 가장 작음
- 넓히면 통로 앞뒤 두 층(파랑)의 가중치가 늘어남 · 잠재 2 202,258 → 128 234,640
- 넓을수록 한 장에 무엇을 남길지 덜 골라도 됨 · 이것은 이번 실행으로 재지 않음

통로 크기는 설계 선택

- 코드에서는 숫자 하나 · `latent_dim`
- 고르는 것은 설계하는 사람
- 고르는 방법: 크기 하나만 바꿔 몇 개 학습 → 원본과 나란히 읽기 → 같은 평가 이미지로 숫자 확인
- MSE 만 보면 128 · 하지만 손글씨 숫자 · 이 구조 · 10 에폭 · 한 번 실행이라는 조건의 결과

다음 실험을 하나만 한다면

- 이 비교를 한 번 더 한다면 무엇 하나를 바꿀까
- 바꾼 뒤 무엇을 보고 나아졌다고 판단할까 · 그림의 어느 칸 · 어떤 숫자
- 힌트: 화면 코드에서 숫자로 적힌 자리들을 보면 바꿀 수 있는 것이 보임
- 바꿀 것 하나 · 판단할 근거 하나

여기까지 질문

- 통로 크기 · `train_ae` · 그림 읽기

정리 — 통로의 크기

- `latent_dim` 숫자 하나로 통로 크기가 바뀜
- 이번 실행: 좁을수록 원본에서 멀어짐 · 8 과 2 에서는 다른 숫자 모양도 나옴 · MSE 도 같은 방향
- 넓히면 MSE 는 작아졌지만 공짜는 아님 · 크기는 쓰임에 맞춰 고르는 설계 선택

실습 P2 — 학습 루프 채우기 · 버그 찾기

PRACTICE_P2.ipynb → 다음 PRACTICE_P2_BUG.ipynb

1. 준비 셀 → 모델 셀 실행 · 모델은 준비된 것(잠재 16)
2. 학습 루프 빈칸: 손실의 두 자리 · `backward()` · `step()` — 함수 안 이름은 `model` · `opt` · 괄호 `()` 까지
3. 실행 → loss 가 내려가고 그림 칸마다 넣은 숫자와 비슷한 모양(흐림)이면 정상 · 막히면 셀 아래 힌트 1 → 2 순서로 펼침
4. P2_BUG(P2-1 을 못 끝냈어도 안내가 나오면 옆): 먼저 실행해 그림만 봄 → 이상한 점 한 줄 → 다른 줄 찾기

P2 확인

```
loss = F.mse_loss(model(x), x)
loss.backward()
opt.step()
```

- 괄호 () 빠뜨리면 에러 없이 학습이 안 됨 · loss 가 꾸준히 내려가지 않음
- 버그판: 정답 자리에 x 가 아닌 따로 섞은 이미지 y · 입력과 정답이 짝이 아님
- 정상판도 흐림 · 차이는 칸마다 다른 숫자 모양(정상) vs 여덟 칸 거의 같은 모양(버그)

다음 시간 질문

잠재 2 모델은 이미지 한 장을 숫자 두 개로 줄임 — 그 두 숫자는 무엇일까

두 숫자를 좌표로 읽기

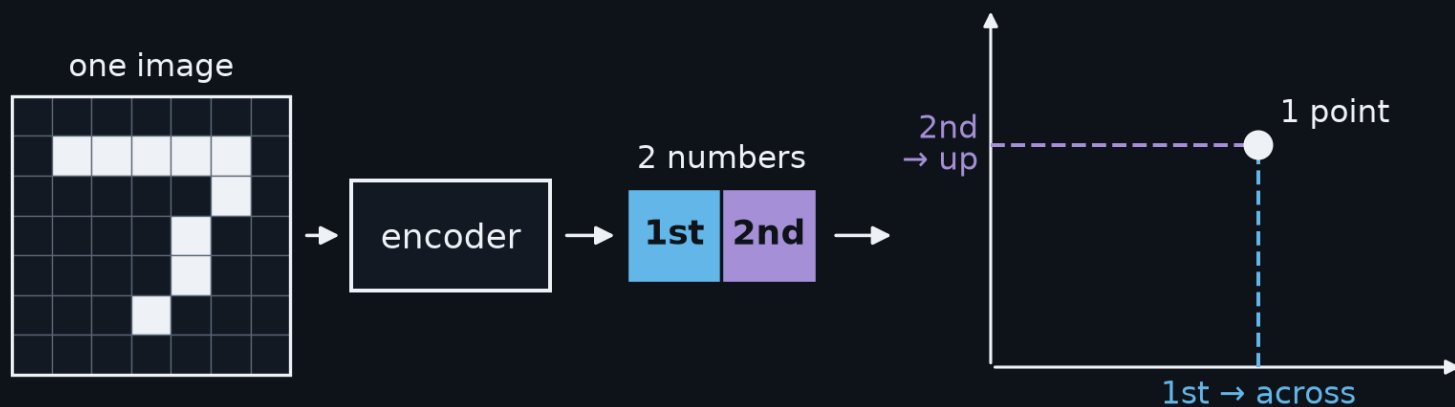
통로에 남은 두 숫자에 무엇이 담겼나

잠재 2 — 아까 본 그림



- latent 2 줄 · 네 모델 중 오차가 가장 컸던 모델

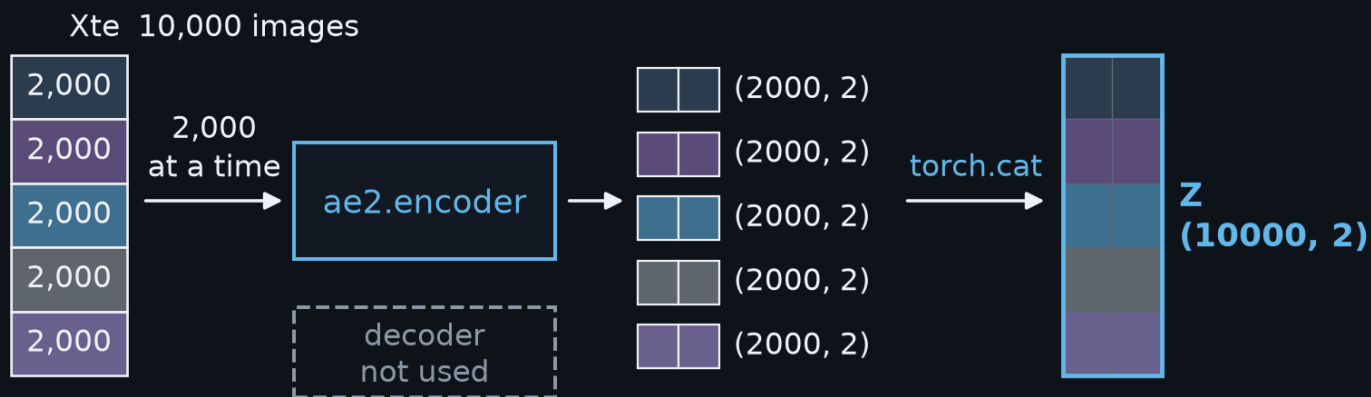
두 숫자를 좌표로



- 첫째 숫자 → 가로 위치 · 둘째 숫자 → 세로 위치
- 평가 이미지 10,000장 → 점 10,000개

인코더만 통과시키기

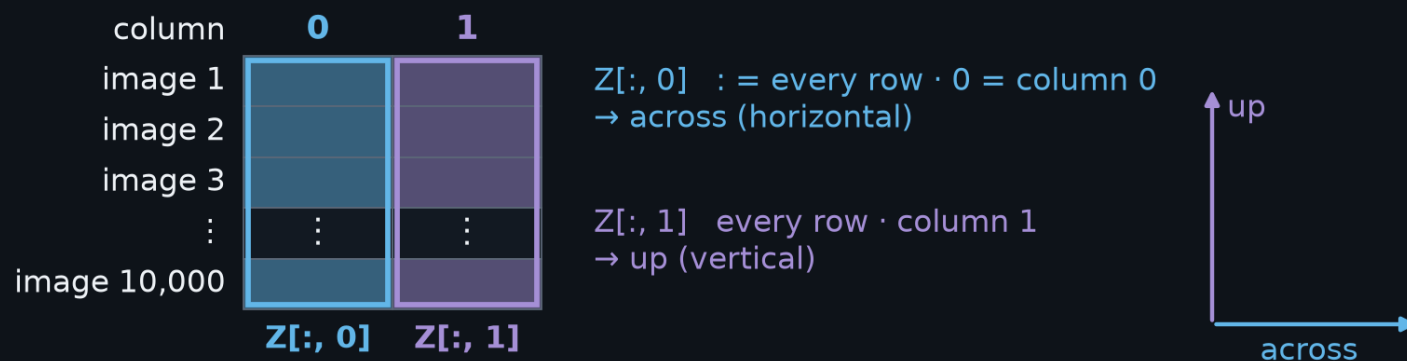
```
ae2 = models[2]
with torch.no_grad():
    Z = torch.cat([ae2.encoder(Xte[i:i + 2000]) for i in range(0, len(Xte), 2000)])
```



- `torch.no_grad()` : 학습이 아니라 꺼내기만 함 · 기울기 계산을 끄

좌표로 찍기

```
plt.scatter(Z[:, 0], Z[:, 1], s=2, c="gray")
```



Z: 10,000 rows (one per image) × 2 columns

- 모든 점이 회색 한 가지 · 라벨 안 씌

찍어 보기

- '5.' 셀
- 출력: **z** 의 모양 한 줄 · 점 그림

출력 첫 줄

잠재 좌표 모양 (10000, 2)

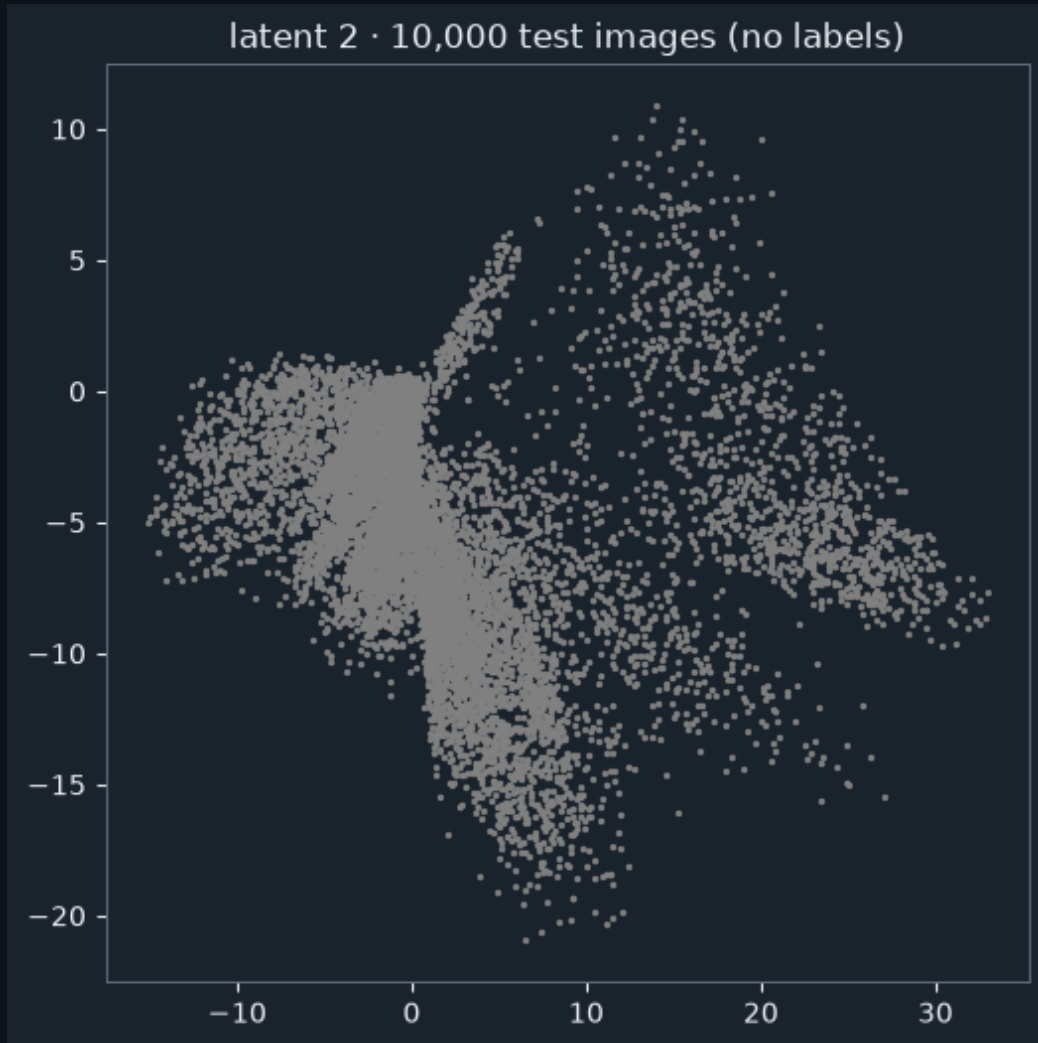
- 10000: 평가 이미지 장수
- 2: 한 장마다 남은 숫자 개수

먼저 세 장만 — 한 장이 점 하나



- 그림마다 인코더가 남긴 숫자 두 개 → (가로, 세로) 좌표
- 한 장 = 점 하나 · 1만 장이면 점 1만 개

점 10,000개 — 색 없음



백백한 곳 · 드문 곳 · 빈 곳



- 백백한 곳: 가운데 띠 · 왼쪽 덩어리 · 드문 곳: 오른쪽 넓은 영역
- 거의 빈 곳: 네 구석 · 가는 줄기와 오른쪽 영역 사이

좌표로 찍은 자리 — 잠재 공간

- 인코더가 내놓은 숫자들을 좌표로 찍은 자리
- 잠재: 겉으로 드러나지 않고 안에 숨어 있음
- 잠재 2 모델 → 평면 · 잠재 32 모델 → 축 32개(그림으로는 못 그림)

질문

학습에 라벨을 안 썼는데, 같은 숫자끼리 가까이 모였을까?

- ① 숫자마다 따로 모임
- ② 뒤섞여 있어 숫자별 모임이 안 보임
- ③ 일부 숫자만 따로 모이고 나머지는 겹침
- 번호 하나 · 이유 하나

라벨은 어디에 있었나

```
yte = te.targets    # 평가 이미지의 숫자 라벨 - 학습에는 쓰지 않음  
loss = F.mse_loss(model(x), x)
```

- `yte`: '0. 준비' 셀에서 불러 둔 라벨 10,000개
- 학습 손실: 복원과 입력 `x` 만 비교 · `yte` 없음

찍은 뒤에 색만 입히기

```
sc = plt.scatter(Z[:, 0], Z[:, 1], s=2, c=yte, cmap="tab10")  
plt.colorbar(sc, ticks=range(10), label="digit label (colored afterwards)")
```

- 같은 `Z` · 점 자리는 회색 그림과 같음
- 바뀐 것: `c=yte` · 라벨 0~9 마다 색 하나
- 오른쪽 색 막대: 어떤 색이 어떤 숫자인지

색 입혀 보기

- '5-1.' 셀

학습이 끝난 뒤 라벨로 색칠



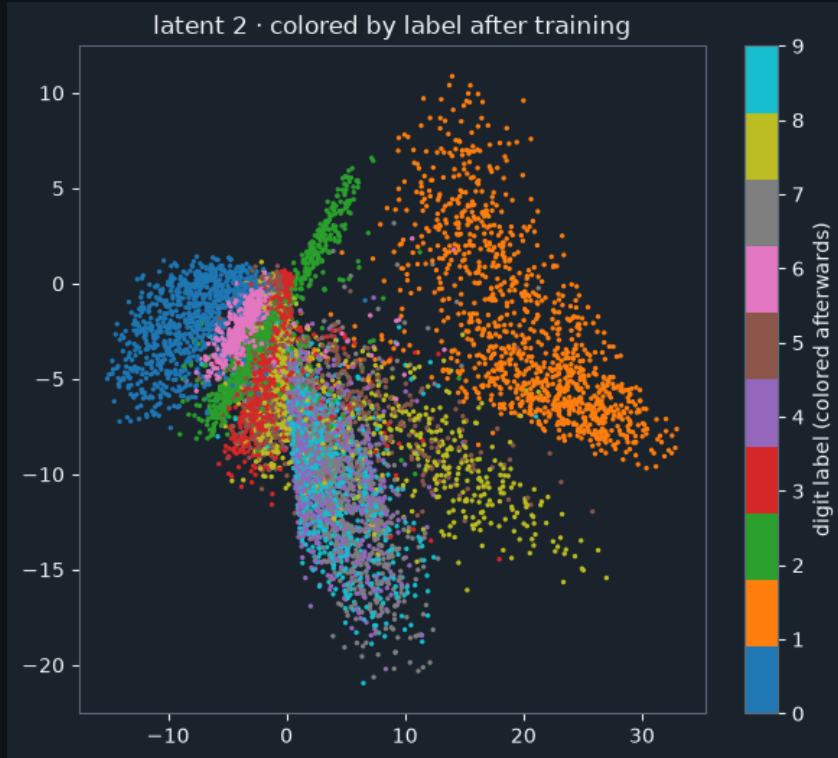
- 색 막대는 숫자와 색의 짝 · 필요할 때 가리킴

따로 자리 잡은 숫자 — 0 · 1 · 2 · 6



- 0(파랑): 왼쪽 넓적한 덩어리 · 6(분홍): 그 오른쪽에 붙은 작고 뾰뾰한 무리
- 1(주황): 오른쪽 넓은 영역 · 다른 색 점은 드문드문
- 2(초록): 오른쪽 위로 뻗은 가는 줄기 · 일부는 가운데 무리 가장자리

겹친 숫자 — 가운데



- 아래로 긴 띠: 4(보라) · 7(회색) · 9(하늘색)가 뒤섞임
- 그 위 왼편: 3(빨강)이 세로로 뻗뻗 · 5(갈색) · 8(연두)이 섞여 흩어짐 · 8 일부는 오른쪽 아래로 뻗음

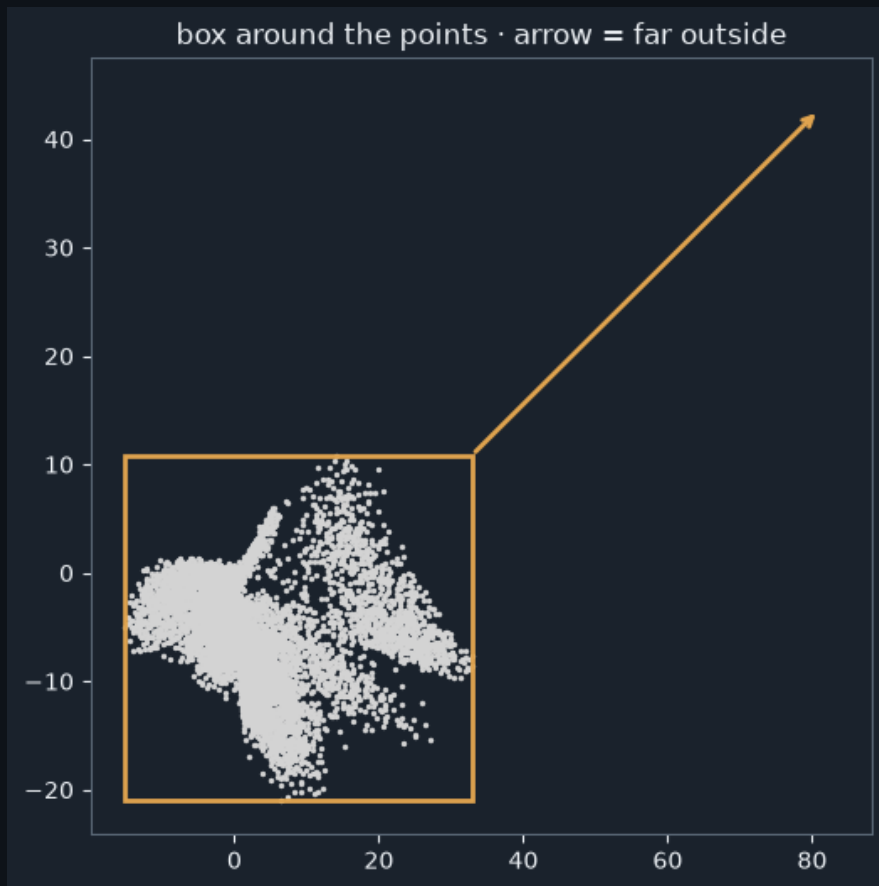
질문의 답 — 이번 실행

- 따로 자리 잡은 숫자: 0 · 1 · 6 · 2 의 상당수
- 겹친 숫자: 4 · 7 · 9 · 그리고 3 · 5 · 8
- 라벨 없이도 일부 숫자는 서로 다른 자리에 놓임

가로축 · 세로축의 뜻

- 축에 뜻을 정해 준 사람 없음
- 모델은 되돌리는 오차가 작아지게 두 숫자를 정했을 뿐
- 축마다 사람 말로 된 이름이 있다고 볼 근거 없음
- 시드를 바꿔 다시 학습한 모양은 확인 안 함

뽕을 곳 두 군데 — 네모 안 · 네모 밖 멀리



- 네모: 찍힌 점들을 둘러싼 범위 · 화살표 끝: 네모 밖 멀리

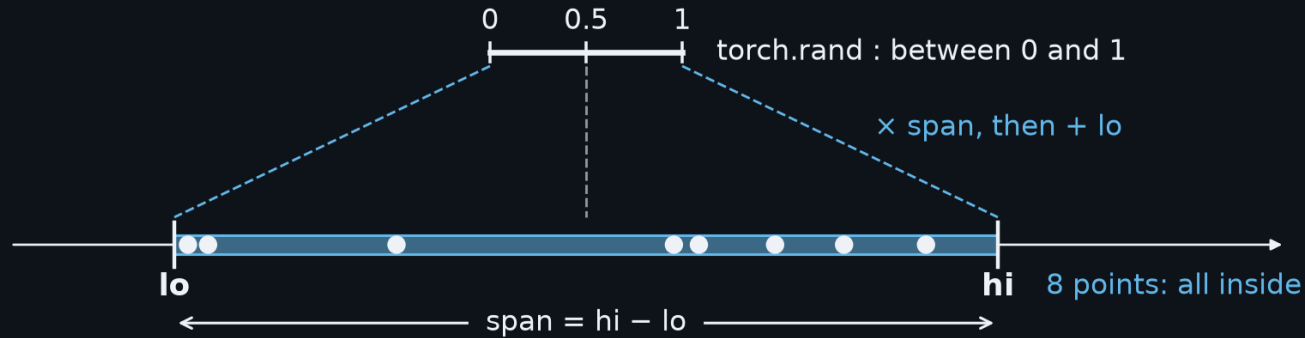
질문

찍힌 점이 없는 곳이나 아주 먼 곳의 좌표를 디코더에 넣으면 무엇이 나올까?

- 가: 점을 둘러싼 네모 안에서 아무렇게나 고른 자리 · 나: 네모 밖 멀리(화살표 끝)
- ① 그래도 숫자처럼 보임 · ② 뭉개진 얼룩 · ③ 알아볼 수 없는 무늬
- 가 · 나 따로 번호 하나씩 · 디코더는 숫자 2개를 받아 아무 좌표나 넣을 수 있음

범위 안에서 뽑기

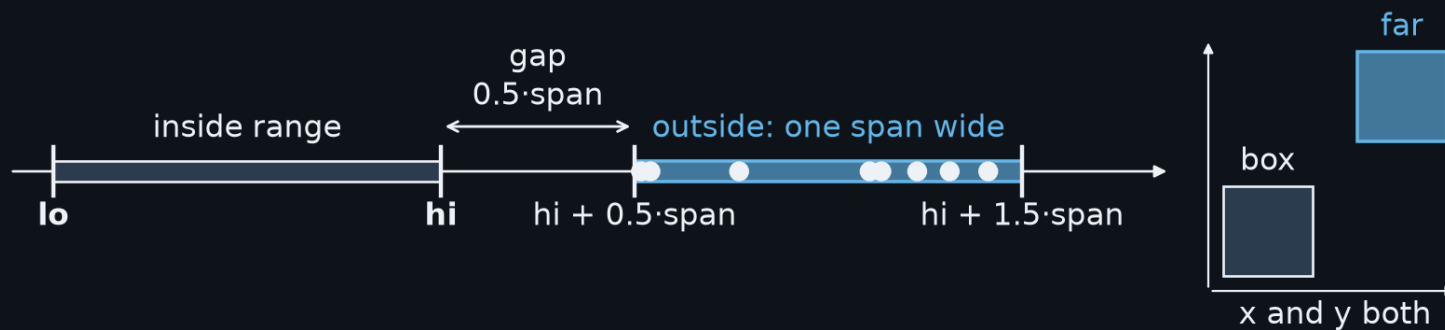
```
lo, hi = Z.min(0).values, Z.max(0).values
span = hi - lo
inside = lo + span * torch.rand(8, 2)
```



- `Z.min(0).values` = 얼마나 가장 작은 값 · `max` 는 가장 큰 값 · 가로 · 세로 따로
- 그림은 한 축 · 가로면 `lo` 왼쪽 끝 · `hi` 오른쪽 끝 · 세로면 아래 끝 · 위 끝
- 네모 안에는 점이 거의 없는 구석도 들어감

범위 밖 멀리 뽑기

```
outside = hi + span * (0.5 + torch.rand(8, 2))
```



- 네모 오른쪽 위 끝(`hi`)에서 폭의 반만큼 띄운 곳부터 폭 하나 길이 구간
- 가로 · 세로 모두 → 산점도로 치면 네모 밖 오른쪽 위 멀리(화살표 끝 쪽)
- 셀 맨 위 `torch.manual_seed(1)` : 다시 실행해도 같은 점

아무 좌표나 넣어 보기

- '5-2.' 셀

출력 맨 아래 줄 — 찍힌 좌표 범위

- 가로 -15.1 ~ 32.9
- 세로 -21.0 ~ 10.9
- 윗줄 8점: 이 상자 안 · 아랫줄 8점: 오른쪽 위로 상자 밖 멀리

윗줄 — random point in range



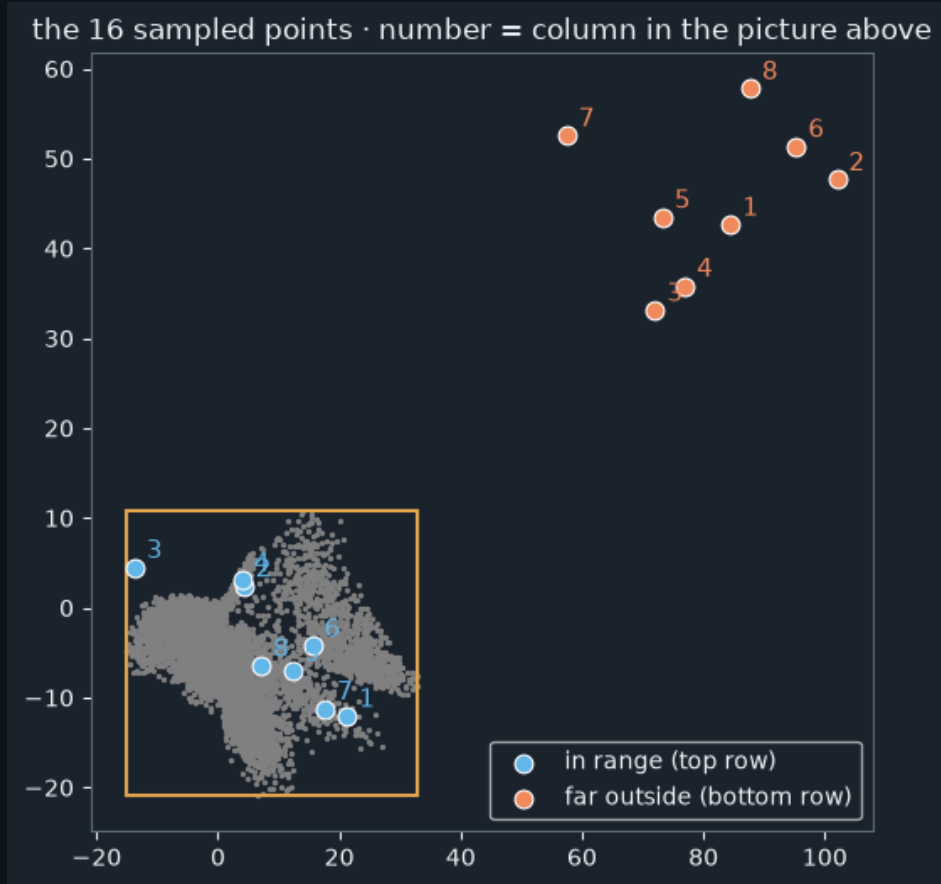
- 왼쪽부터: 8 · 2 · 굵고 밝은 0 · 2 · 흐린 8 · 흐린 1 · 8 · 흐려서 8 인지 9 인지 애매
- 이번 8점은 모두 숫자처럼 보임

아랫줄 — far outside the range



- 8장 모두 거의 같은 모양: 오른쪽으로 기운 굵은 획 하나
- 윗줄보다 훨씬 밝고 가장자리가 각짐
- 7번째만 조금 가늘고 오른쪽 아래에 작은 점

뽕은 16점은 어디에 떨어졌나



- 파랑 = 윗줄(상자 안) · 주황 = 아랫줄(상자 밖) · 번호 = 그림의 칸 순서
- 파랑 3번(밝고 붉은 0)만 회색 점이 거의 없는 곳 · 나머지 파랑은 점이 모인 곳 근처

아무 좌표나 넣었을 때 — 이번 실행

	뽑은 곳	나온 그림
윗줄	찍힌 범위 상자 안	8장 모두 숫자처럼 보임
아랫줄	상자 밖 오른쪽 위 멀리	거의 같은 굵은 획 8장

- 윗줄 8점 대부분은 점이 모인 곳 근처 · 빈 곳에 떨어진 1점은 밝고 굵은 0

내일로 넘기는 질문

어디서 뽑아야 그럴듯한가

- 오늘은 답하지 않음

두 숫자를 좌표로 읽기 — 정리

한 것	이번 실행에서 본 것
10,000장을 두 숫자로 찍음	여러 갈래로 뺀 모양 · 뾰뾰한 곳 · 빈 곳
찍은 뒤 라벨로 색칠	0 · 1 은 따로 · 4 · 7 · 9 는 겹침
아무 좌표나 넣기	상자 안 8점은 숫자처럼 · 멀리는 같은 굵은 획

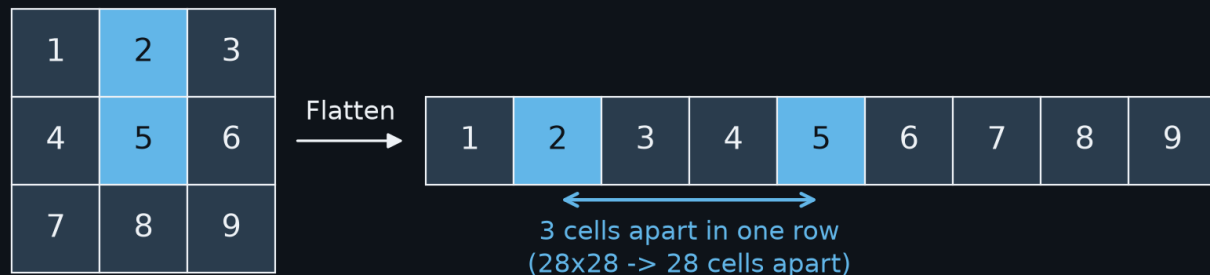
질문

- 산점도 · 색칠한 산점도 · 아무 좌표 — 세 그림 중 궁금한 것

지금까지 모델의 입구

```
nn.Flatten(), nn.Linear(784, 128)
```

3x3 image: 2 and 5 are neighbors (above / below)



- 28×28 이미지 → 784개 숫자 한 줄로 펴
- 위아래로 붙어 있던 두 픽셀 → 한 줄에서는 28칸 떨어짐

쉬고 와서 할 질문

이미지를 퍼지 않고, 이미지 모양 그대로 줄였다가 되돌릴 수 있을까?

- CNN 과정: `Conv2d` 로 이미지를 퍼지 않고 다룸

이미지를 이미지 모양 그대로

늘리는 합성곱과 크기 맞추기

미션 5 가 요구하는 모델

미션 안내 · 모델 설계

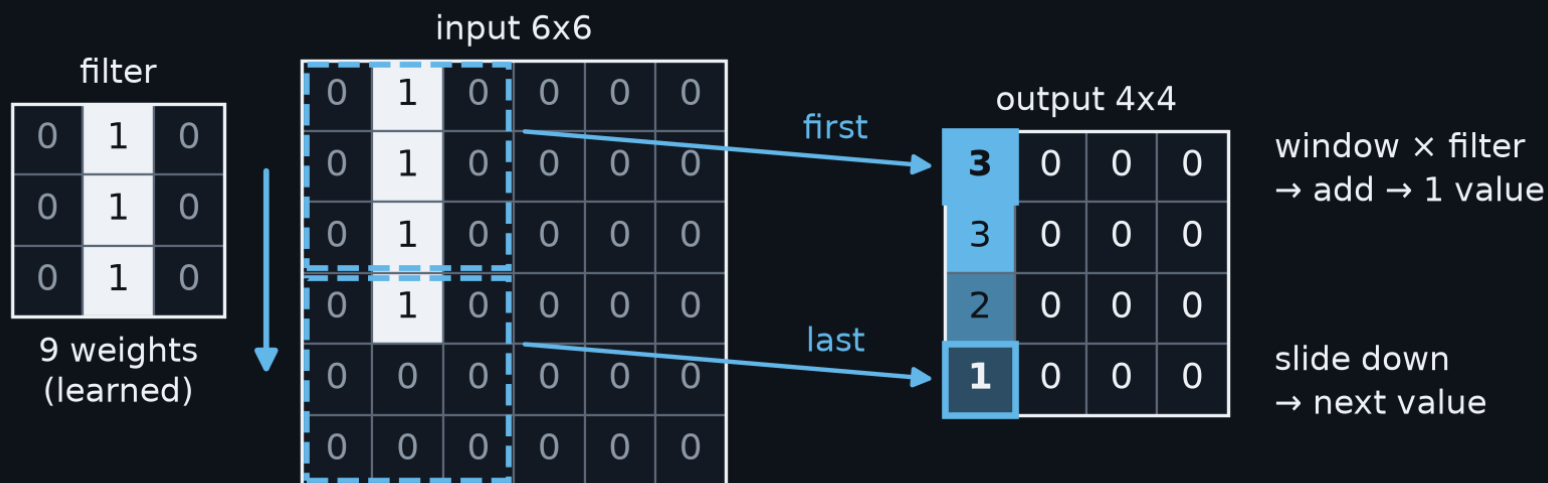
`Conv2d` 로 인코더를, `ConvTranspose2d` 로 디코더를 구성한 CNN 오토인코더를 설계하세요.

- 인코더 = 줄이는 쪽 · `Conv2d`
- 디코더 = 되돌리는 쪽 · `ConvTranspose2d`
- 같은 안내가 꼽은 가장 자주 만나는 에러: 인코더 · 디코더 크기 어긋남

이번 시간 순서

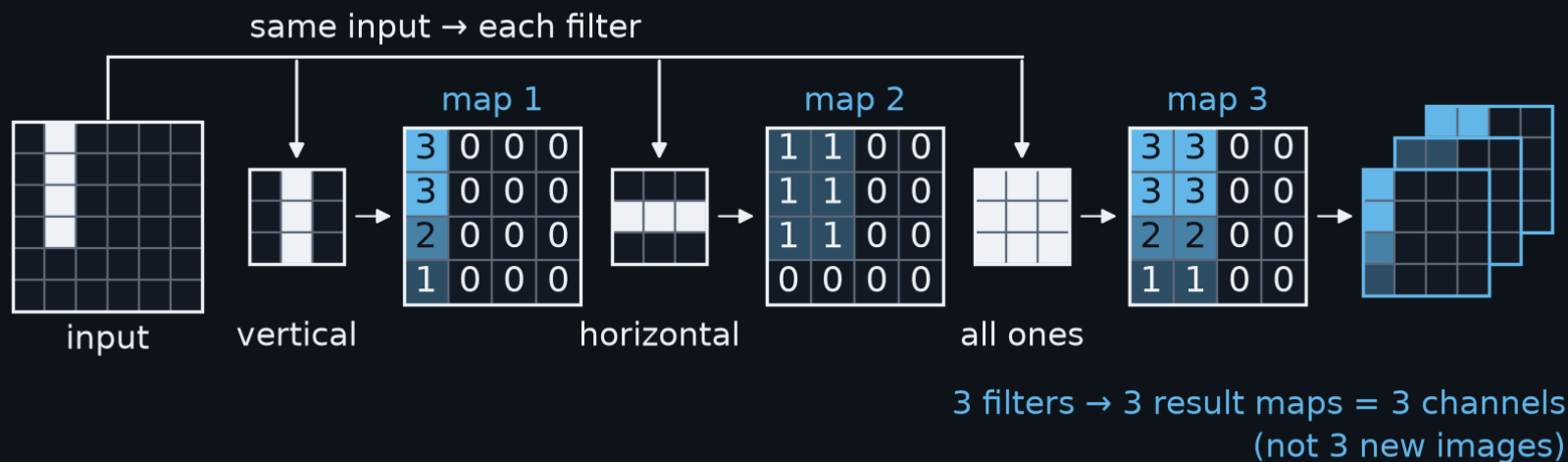
순서	할 것	층
0	합성곱이 하는 일	필터 · 채널
1	줄이기	Conv2d
2	늘리기	ConvTranspose2d
3	늘린 크기 따져 보기	ConvTranspose2d
4	여러 크기 넣어 보기	줄였다 늘리는 작은 모델

합성곱이 하는 일 — 필터 하나



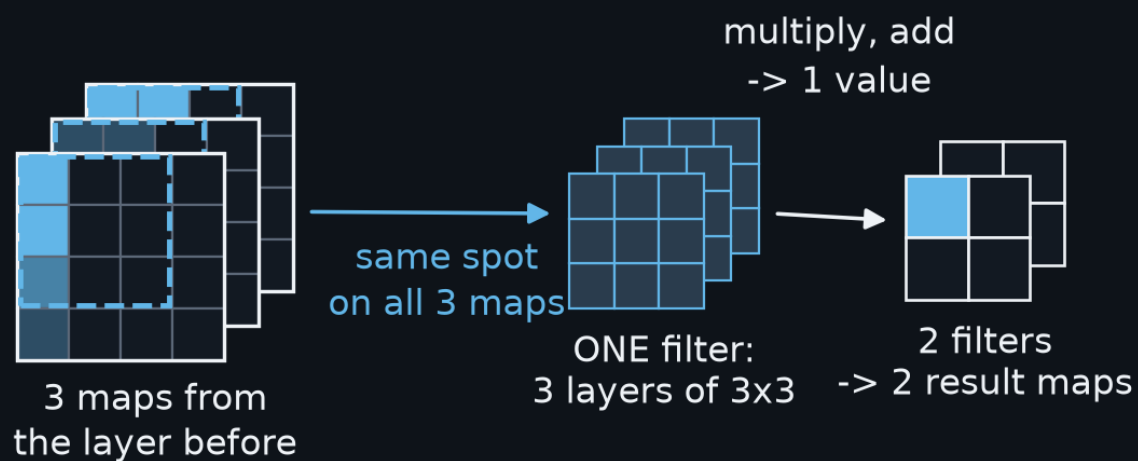
- 3×3 필터: 가중치 9개 · 학습으로 정해짐
- 한 자리의 3×3 칸과 곱해 더함 → 출력 값 하나 · 옮겨서 같은 필터로 다시 계산

필터 여러 개 → 결과 판 여러 장 = 채널



- 필터마다 가중치가 달라 서로 다른 결과 판이 나옴
- 결과 판의 수 = 채널 수 · 입력 이미지가 늘어난 것이 아님

다음 층의 필터 — 들어온 판을 전부 한꺼번에 봄



Conv2d(3, 2, 3)

weight shape [2, 3, 3, 3]

= [filters, maps in, 3, 3]

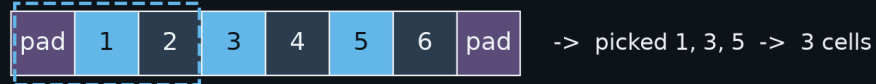
today: Conv2d(16, 32, 3)

-> weight [32, 16, 3, 3]

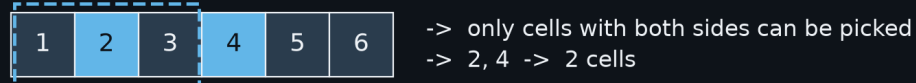
- 필터 하나: 들어온 판 3장의 같은 자리를 함께 곱해 더함 → 값 하나
- 필터 하나 = 3x3 가중치가 판 수만큼 곱 · 필터 수 = 새 결과 판 수
- Conv2d(16, 32, 3) : 필터 하나가 16장을 봄 · 필터 32개 → 가중치 모양 [32, 16, 3, 3]

Conv2d 로 줄이기 — 한 칸 건너 하나씩

padding 1 : pad cells on both ends



padding 0 : no pad



dashed box = kernel 3 window (picked cell + both sides)

- stride 2: 한 칸 건너 하나씩 고름
- kernel 3: 고른 칸과 양옆 한 칸씩을 같이 봄
- padding 1: 양쪽 가장자리 옆에 한 칸씩 덧댄 · 첫 칸 · 끝 칸도 양옆을 볼 수 있게
- 고른 칸 수 = 출력 크기

생각해 볼 것

```
Conv2d(kernel_size=3, stride=2, padding=1)
```

- 28×28 을 넣으면 크기는? · 그걸 한 번 더 넣으면?
- 한 칸 건너 하나씩 고르는 그림으로

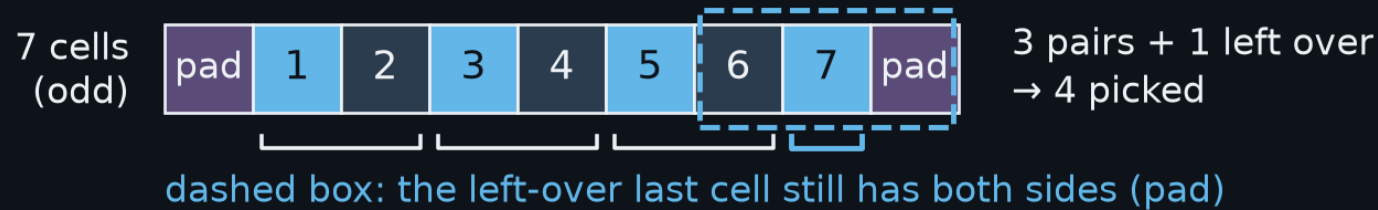
실행해서 확인

- '6. 합성곱으로 줄이기' 셀
- 28×28 을 Conv2d 에 한 번 · 그 결과를 한 번 더

28 → 14 → 7

- 셀 출력 두 줄: $28 \times 28 \rightarrow 14 \times 14$ · 한 번 더 $\rightarrow 7 \times 7$
- 28 은 짝수 $\rightarrow$ 딱 반
- 14 도 짝수 $\rightarrow$ 또 딱 반

칸 수가 홀수라면

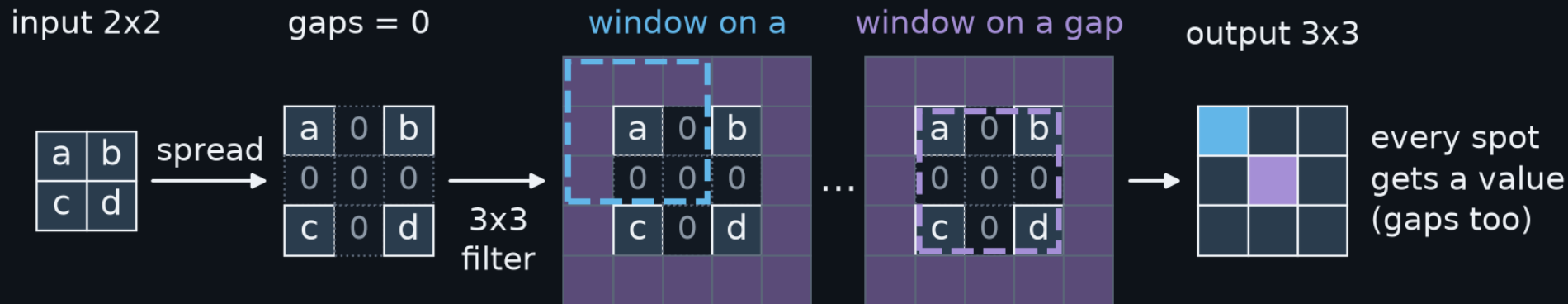


- 둘씩 묶으면 쌍마다 한 칸씩 뽑힘 · 남는 끝 칸도 뽑힘
- 짝수 칸: 쌍 개수만큼 = 딱 반 · 홀수 칸: 쌍 개수 + 1
- 그림으로 센 것 · 셀로 확인한 것은 짝수(28 · 14)뿐

이제 반대 방향

- 인코더: $28 \rightarrow 14 \rightarrow 7$
- 디코더: $7 \rightarrow ? \rightarrow 28$
- 줄이는 데 쓴 Conv2d 말고 늘리는 층이 필요함

늘리는 합성곱 — 칸 사이를 벌림



■ pad on the edge · window moves 1 step at a time · a picture for counting size

- 칸 사이를 벌림 → 틈을 0 으로 채움 → 그 위에 합성곱 · 틈 자리 값까지 새로 계산
- stride: 원래 칸들을 몇 칸 간격으로 놓을지 · stride 2 면 두 칸 간격 → 칸 사이에 틈 하나
- 줄일 때는 ■□■□ 에서 ■ 만 고름 · 늘릴 때는 ■ 사이에 □ 를 끼움

전치 합성곱 · ConvTranspose2d

- 칸 사이를 벌리고 합성곱해 크기를 늘리는 층
- 이름: 전치 합성곱 · PyTorch 에서는 `ConvTranspose2d`
- Conv2d 와 같은 인자: `kernel_size` · `stride` · `padding`
- 가중치는 이 층이 따로 학습함

예측 ③

`ConvTranspose2d(kernel_size=3, stride=2, padding=1)` 로 32×32 를 늘리면 몇×몇?

- 칸 사이를 벌리는 그림으로 생각해 봄
- 이유 하나

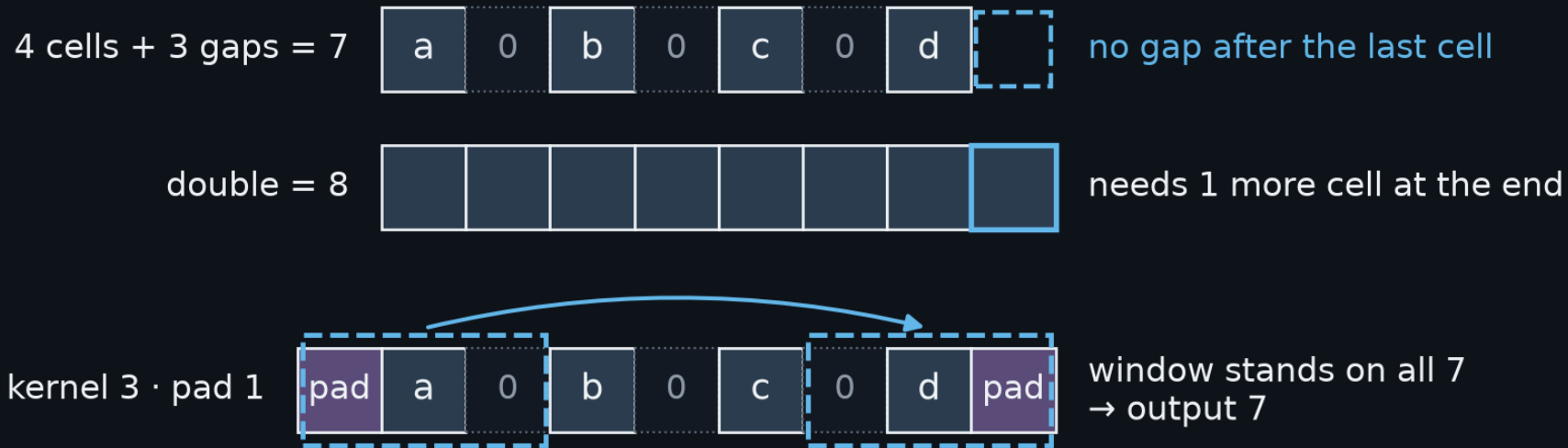
실행해서 확인

- '6-1. 전치 합성곱으로 늘리기' 셀
- 32×32 · 채널 16 짜리 빈 텐서 → ConvTranspose2d k3 s2 p1

$32 \times 32 \rightarrow 63 \times 63$

- 딱 두 배가 아님
- 한 칸 모자람

그림으로 — 칸 32개 · 틈 31개



- 틈은 칸 사이에만 생김 · 맨 끝 칸 뒤에는 없음
- kernel 3 · padding 1: 창이 한 칸씩 옮기며 모든 칸에 섬 → 벌린 크기 그대로
- 칸 32 + 틈 31 → 63 · 두 배가 되려면 맨 끝에 한 칸이 더 있어야 함

한 칸이 왜 문제인가

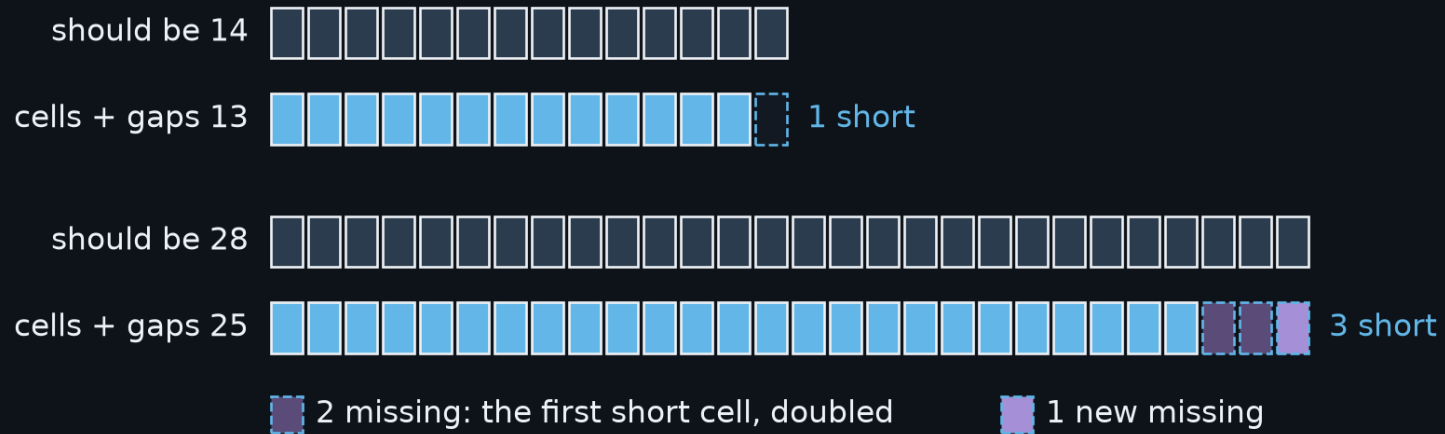
- 디코더 출력은 원본과 크기가 같아야 함
- 손실: 복원과 원본을 칸마다 비교
- 크기가 다르면 칸마다 짝이 맞지 않음

생각해 볼 것 — 두 번 늘리면

`ConvTranspose2d(kernel_size=3, stride=2, padding=1)` 두 번

- 한 번 늘릴 때 한 칸 모자람
- 두 번 거치면 모자란 칸은 한 칸 그대로일까, 더 커질까?
- 이유 하나

모자람이 쌓임



- 칸 + 틈으로 셈 · 첫 번째에서 빈 한 칸이 두 번째에서 벌어져 두 칸 · 여기에 새로 한 칸 더 → 세 칸
- 빈 칸 색 두 가지: 첫 번째 빈 칸이 벌어진 것 · 새로 빈 것
- 아직 실행하지 않음

생각해 볼 것 — 한 칸 채우기

$32 \times 32 \rightarrow 64 \times 64$ 로 딱 두 배

- 모자란 곳: 맨 끝 한 칸
- stride 는 2 그대로 · 무엇을 바꾸면 한 칸이 더 생길까?

실행해서 확인

- '6-2. 딱 두 배로 맞추기' 셀
- 같은 32×32 를 두 가지 층에 넣음

방법 1 · output_padding=1 → 64×64

- `output_padding=1` : 출력 크기를 한 칸 크게 정하는 인자(기본값 0) · 0 을 덧붙이는 것 아님 · 늘어난 칸도 필터로 계산
- 다른 설정은 그대로 · 한 칸 모자라던 63 → 64
- 칸 + 틸은 늘 두 배보다 한 칸 적음 → 이 방법이면 어떤 크기든 딱 두 배 · 다음 시간 Conv 오토인코더가 씬

방법 2 · kernel_size=4 → 64×64

- kernel 4 · stride 2 · padding 1 · output_padding 없음
- kernel 을 한 칸 넓히니 모자란 한 칸이 채워져 딱 두 배
- 칸 + 틈은 늘 두 배보다 한 칸 적음 → 이 층도 어떤 크기든 딱 두 배 · 셀로 본 것은 32×32 · 공식은 학생판 읽을거리

작은 모델 TinyConvAE — '7.' 셀

```
self.encoder = nn.Sequential(
    nn.Conv2d(1, 8, 3, stride=2, padding=1), nn.ReLU(),
    nn.Conv2d(8, 16, 3, stride=2, padding=1))
self.decoder = nn.Sequential(
    nn.ConvTranspose2d(16, 8, 4, stride=2, padding=1), nn.ReLU(),
    nn.ConvTranspose2d(8, 1, 4, stride=2, padding=1))
```

- `nn.Conv2d(받는 채널, 내보내는 채널, kernel, stride=..., padding=...)` · 셋째 숫자가 `kernel` · 크기만 보는 모델이라 채널 숫자는 신경 쓰지 않아도 됨
- 인코더: `Conv2d k3 s2 p1` 두 번 · 디코더: `ConvTranspose2d k4 s2 p1` 두 번
- 학습하지 않음 · 크기만 봄

무엇이 나올까

- 넣을 크기: $28 \times 28 \cdot 32 \times 32 \cdot 30 \times 30 \cdot 250 \times 536$
- 먼저 28 과 30: 넣은 크기 그대로 안 돌아올 것은?
- 반으로 줄일 때마다 짝수인지 홀수인지 따라가 봄
- $32 \cdot 250 \times 536$ 은 결과를 본 뒤 같이 따라감
- 이유 하나

실행해서 확인

- '7. 여러 크기 넣어 보기' 셀
- 네 크기의 출력 · 이어서 30×30 으로 손실 계산

출력 크기 — 셀 출력 위 네 줄

입력 28×28 → 출력 28×28

입력 32×32 → 출력 32×32

입력 30×30 → 출력 32×32 ← 크기가 다름

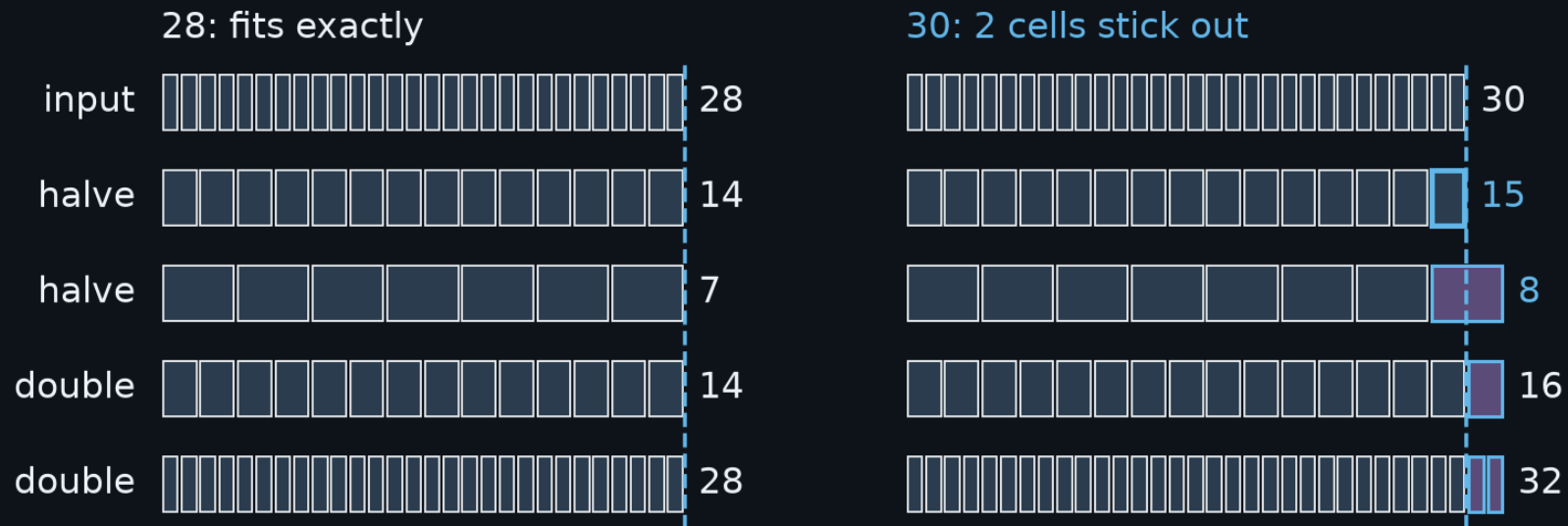
입력 250×536 → 출력 252×536 ← 크기가 다름

손실 계산 에러 — 셀 출력 마지막 줄

손실 계산 에러: The size of tensor a (32) must match the size of tensor b (30) at non-singleton dimension 3

- a (32): mse_loss 첫 인자 = 모델 출력의 가로 · b (30): 입력의 가로
- dimension 3: 자리 번호는 0 부터(0 장 수 · 1 채널 · 2 세로 · 3 가로) → 가로
- 세로도 32 와 30 으로 다르지만 메시지는 한 자리만 알려 줌

30×30 은 왜 32×32 가 됐나



same width scale: 1 halved cell = 2 original cells wide · dashed line = original width

- 15 홀수 → 남는 끝 칸도 뽑혀 8 · 줄일 때 한 칸 더 뽑힌 만큼 늘리고 나면 원래보다 커짐
- 가운데 크기는 칸 고르기로 센 값 · 셀이 찍은 것은 마지막 크기

250×536 — 짝수 · 홀수로 따라가기

단계	세로 250	가로 536
반으로 1번	250 짝수 → 125	536 짝수 → 268
반으로 2번	125 홀수 → 쌍 개수 + 1 = 63	268 짝수 → 134
두 배로 2번	63 → 126 → 252	134 → 268 → 536

- 가로는 두 번 모두 짝수 → 그대로 · 세로만 어긋남
- 가운데 크기는 칸 고르기로 센 값 · 셀이 짝인 것은 마지막 크기

넣어 본 네 크기에서 본 것

입력	반으로 두 번 다 짝수?	돌아왔나
28×28	예	예
32×32	예	예
30×30	아니오 · 15 가 홀수	아니오 · 32×32
250×536	세로 아니오 · 125 가 홀수 / 가로 예	세로만 어긋남 · 252×536

- ✓



28 cells = 7 groups of 4
- ✗



30 cells = 7 groups of 4 + 2 left over
- ✓



32 cells = 8 groups of 4

halve twice and come back exactly
⇒ splits into groups of 4 with nothing left over

- 4칸씩 묶어 남는 칸이 없으면 돌아옴(반으로 두 번 접어도 남는 칸 없음) · 536 은 안 남음 · 250 · 30 은 남음

정리 — 층마다 본 크기

층	크기
Conv2d k3 s2 p1	28 → 14 → 7 · 딱 반
ConvTranspose2d k3 s2 p1	32 → 63 · 한 칸 모자람
+ output_padding=1 · 또는 k4 s2 p1	32 → 64 · 딱 두 배

- 크기는 짝수 · 홀수 · 칸과 틈으로 따져 보고 출력 모양으로 확인

```
nn.Conv2d(1, 16, 3, stride=2, padding=1)    # 1×28×28 → 16×14×14
```

검사 도구 출력 읽는 법 — 예시

예시 · 실습과 다른 모델 · 입력 32×32

1번째	층	표	16	찍힌	16	
2번째	층	표	8	찍힌	8	
3번째	층	표	16	찍힌	15	← 여기서 처음 다름
4번째	층	표	32	찍힌	30	

표와 다른 층이 있음 → 처음 다른 층의 설정을 표와 비교해 한 곳만 고침

- ← 표시가 붙은 줄만 봄 · 그 층의 설정을 표와 비교
- [표 먼저] 가 찍히면 코드보다 표부터 다시 셈
- 통과 가 찍히면 끝

실습 P3 — 크기 표 채우기 · 어긋난 층 찾기

PRACTICE_P3.ipynb

1. 준비 셀 · 검사 도구 셀 실행 → 크기 표 빈칸을 칸 고르기 · 칸과 틸드로 채움
2. 표의 '나오는 크기' 앞 세 칸을 셀의 `표 = [____, ____, ____, 28]` 에 옮겨 적고 실행 →
표가 틀리면 표부터 고치라고 알려 줌
3. 표와 찍힌 크기 비교 → 처음 달라지는 층의 설정 한 곳만 고침
4. `통과` 가 찍히면 끝 · 크기만 맞추고 설정이 표와 다르면 통과 아님
 - 찍힌 모양 (장 수, 채널, 세로, 가로) · 끝 두 숫자가 크기

P3 확인

층	들어가는 크기	나오는 크기
Conv2d k3 s2 p1	28	14
Conv2d k3 s2 p1	14	7
ConvTranspose2d k3 s2 p1 op1	7	14
ConvTranspose2d k3 s2 p1 op1	14	28

- 찍힌 모양 $14 \rightarrow 6 \rightarrow 12 \rightarrow 24$ · 둘째 Conv2d 에서 처음 달라짐
- 그 층 설정 padding=0 → padding=1 로 고치면 28 로 돌아옴
- padding 0: 양옆이 다 있는 칸만 고를 수 있음 → 14칸에서 ②④⑥⑧⑩⑫ = 6

이미지를 이미지 모양 그대로 줄였다가 되돌리기

Conv 오토인코더

오전의 AE — 입력을 한 줄로 펴

단계	크기
입력	1×28×28
<code>nn.Flatten()</code>	784
<code>Linear</code> · <code>Linear</code>	128 → 32
<code>Linear</code> · <code>Linear</code>	128 → 784
<code>reshape</code>	1×28×28

- 이웃한 픽셀이라는 배치를 쓰지 않고 784개 값을 한 줄로 받음

Conv 오토인코더

- 줄이는 쪽(인코더): `Conv2d`
- 되돌리는 쪽(디코더): `ConvTranspose2d`
- 할 일은 AE 와 같음: 입력을 줄였다가 입력 자신으로 되돌림

치기 전에 설계부터 — 인코더

층	나오는 크기 (채널×세로×가로)
입력	1×28×28
<code>Conv2d(1, 16, 3, stride=2, padding=1)</code>	16×14×14
<code>Conv2d(16, 32, 3, stride=2, padding=1)</code>	32×7×7

- 층을 지날 때마다 세로 · 가로는 반 · 채널은 늘림

채널 16 · 32 는 어디서 왔나

- 채널 = 필터 수 · 필터마다 결과 판 한 장
- 고르는 자리: 각 층이 내보내는 채널 수($16 \cdot 32$) · 이 노트북에서 정함
- 따라가는 자리: 다음 층이 받는 채널 수 = 앞 층이 내보낸 수
- 세로 · 가로 줄이는 대신 채널을 늘리는 모양

디코더는 거꾸로

층	나오는 크기
가운데	$32 \times 7 \times 7$
?	$16 \times 14 \times 14$
?	$1 \times 28 \times 28$

- $7 \times 7 \rightarrow 14 \times 14 \cdot 14 \times 14 \rightarrow 28 \times 28$: 딱 두 배
- 어떤 층 · 어떤 설정?

디코더 설계 — ConvTranspose2d + output_padding

층	나오는 크기
가운데	32×7×7
<code>nn.ConvTranspose2d(32, 16, 3, stride=2, padding=1, output_padding=1)</code>	16×14×14
<code>nn.ConvTranspose2d(16, 1, 3, stride=2, padding=1, output_padding=1)</code>	1×28×28

- 쉬기 전 방법 1: 칸 + 틈으로 한 칸 모자람 → output_padding=1 로 출력을 한 칸 크게
- 7 → 14 → 28
- 인자 순서: 받는 채널 · 내보내는 채널 · kernel · ...

마지막 층 · 그리고 28 이 맞아떨어지는 이유

- 마지막 층 채널 1: 입력과 같은 흑백 한 장 · 그래야 입력과 MSE 로 비교
- 마지막에 `Sigmoid` : 출력을 0~1 로 · 오전 AE 와 같음
- $28 \rightarrow 14 \rightarrow 7 \rightarrow 14 \rightarrow 28$: 두 번 모두 짝수가 들어가 딱 반씩 줄었음
- 쉬기 전에 넣은 30×30 은 되돌아오지 않았음: 32×32

실습 P4 — Conv 디코더 채우기

PRACTICE_P4.ipynb

1. 준비 셀 실행 · 디코더 뼈대 `nn.ConvTranspose2d(____, ____, 3, stride=2, padding=1, output_padding=1)`
2. 빈칸: 받는 채널 · 내보내는 채널 숫자 네 개 · 마지막 함수 이름 하나
3. 확인 셀 실행 → 마지막 모양 (1, 1, 28, 28) · 층 목록에 ReLU · Sigmoid
4. 맨 아래 1 에폭 학습 셀 실행 → 내가 만든 모델의 복원 그림 확인

P4 확인 — 노트북 '8. Conv 오토인코더'

```
self.decoder = nn.Sequential(  
    nn.ConvTranspose2d(32, 16, 3, stride=2, padding=1, output_padding=1), nn.ReLU(),  
    nn.ConvTranspose2d(16, 1, 3, stride=2, padding=1, output_padding=1), nn.Sigmoid())
```

- 채널 $32 \rightarrow 16 \rightarrow 1$ · 인코더를 거꾸로
- 끝의 `))` : `Sigmoid()` 와 `nn.Sequential()` 을 함께 닫음

forward — 펴지도 되돌려 접지도 않음

```
def forward(self, x):  
    return self.decoder(self.encoder(x))
```

- 오전 AE: 끝에 `.reshape(-1, 1, 28, 28)` 로 다시 접었음
- ConvAE: 처음부터 끝까지 채널×세로×가로 모양이 유지됨

실행 전에 하나

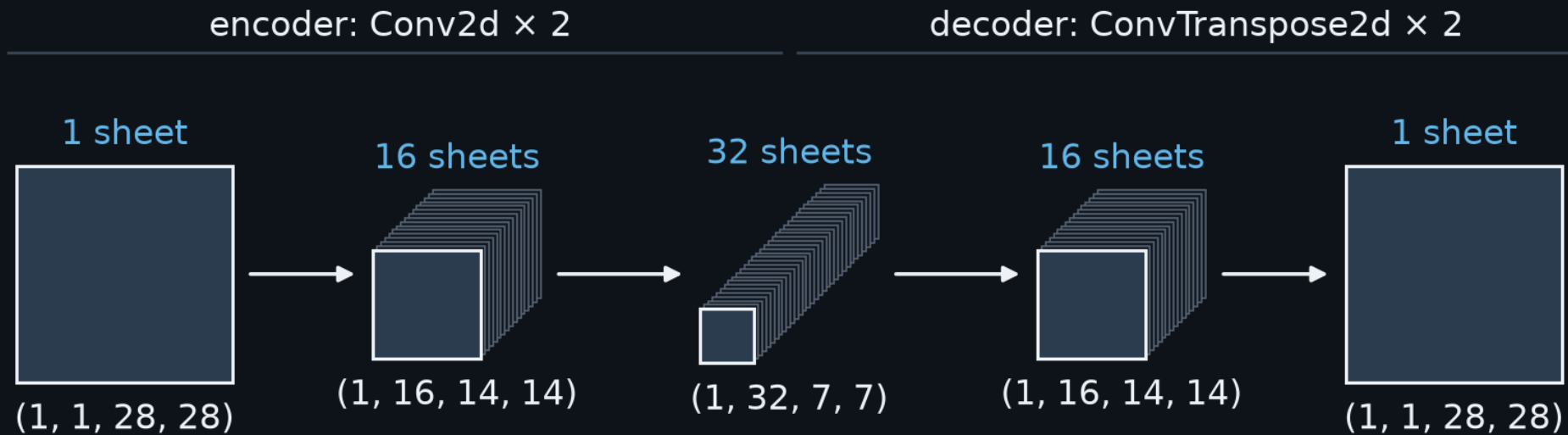
가운데 통로 $32 \times 7 \times 7$ 에 담기는 값 개수 · 입력 값 784개보다 좁을까, 넓을까?

- 좁음 · 비슷함 · 넓음
- 답 하나 · 이유 하나

실행 — 층마다 모양 찍기

- 입력 한 장을 층마다 통과시키며 모양을 찍음
- 맨 아래 줄: 파라미터 수(학습으로 고쳐지는 가중치 개수)

찍힌 모양



(images, channels, height, width) · sheets = channels · face = height × width

- 괄호 = 셀이 찍은 모양 · 맨 앞 1: 이미지 한 장 · 나머지는 설계 표와 같음
- 판 수 = 채널 · 앞면 = 세로 × 가로

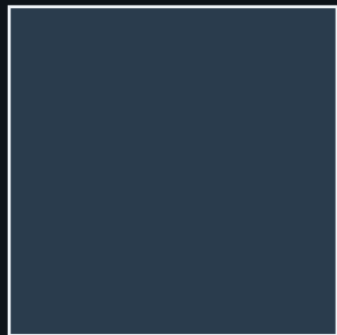
모양은 이렇게 찍었음

```
h = Xte[:1]
for layer in cae.encoder:
    h = layer(h)
```

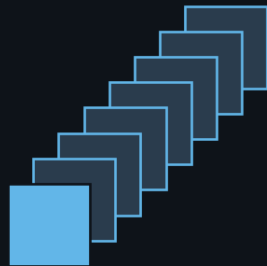
- 이미지 한 장을 층 하나씩 통과시킴
- Conv2d 를 지난 뒤마다 h.shape 를 찍음 · 디코더도 같은 방식
- 크기가 어긋날 때 어느 층에서 달라지는지 찾는 방법

가운데 통로 — 입력보다 넓은

	값 개수
입력 $1 \times 28 \times 28$	784
가운데 $32 \times 7 \times 7$	1,568



input: 1 map, 28×28

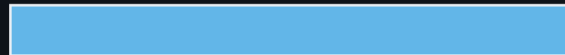


middle: 32 result maps
(one per filter), each 7×7

input 784 values



middle 1,568 values



values (same scale)

오전 AE 의 통로와 다름

모델	가운데 값 개수	입력 값 784개와 비교
AE	32	좁음
ConvAE	1,568	넓음

- 값 개수로는 입력 784개를 다 담고도 남음
- 무엇을 남기고 버렸는지는 이 셀로는 모름

생각해 볼 것 — 통로를 입력만큼으로

- 지금 가운데는 입력의 두 배
- 세로 · 가로는 7×7 그대로 · 인코더 마지막 `Conv2d` 의 채널만 바꾼다면
- 채널을 몇으로 하면 입력과 같은 값 개수?

채널 × 세로 × 가로

마지막 Conv2d 채널	가운데 값 개수	입력 값 784개와 비교
32	$32 \times 49 = 1,568$	두 배
16	$16 \times 49 = 784$	같음
8	$8 \times 49 = 392$	반

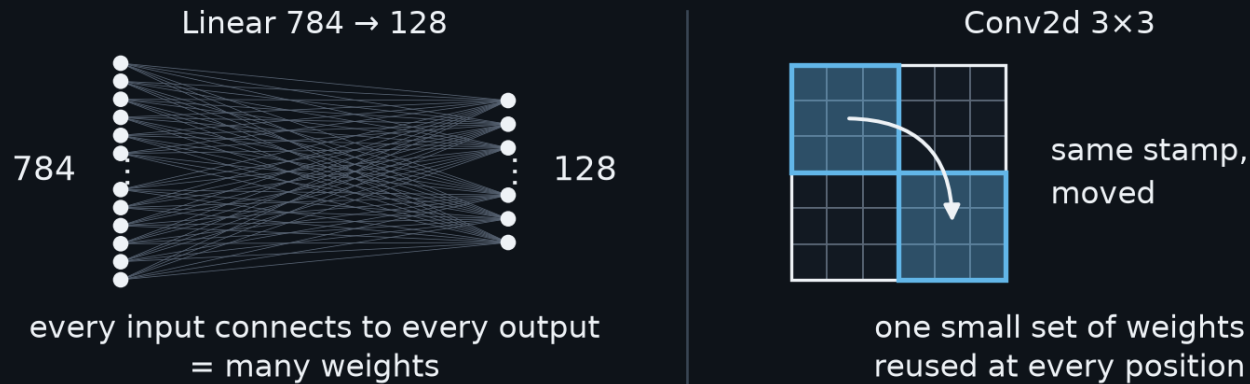
- 곱셈으로 센 값 · 채널을 바꿔 학습해 보지는 않음
- 채널을 바꾸면 디코더 첫 층이 받는 채널도 같이 바뀜

파라미터 수 — 셀 출력 맨 아래 줄

모델	파라미터
AE(잠재 32)	209,968
ConvAE	9,569

- 가운데 통로는 ConvAE 가 훨씬 넓은데 파라미터는 훨씬 적음
- 왜일까? · 두 층이 이미지를 보는 방식에서

연결하는 방식이 다름



values (bowl) $\neq$ weights (knobs the training turns)

- Linear: 모든 입력이 모든 출력과 이어짐 → 가중치 많음
- Conv: 작은 도장 하나를 옮겨 가며 다시 씌움 · 이미지가 커져도 가중치 수 그대로

일부러 틀려 보기 ①

```
nn.ConvTranspose2d(16, 16, 3, stride=2, padding=1, output_padding=1) # 첫 인자를 32 대신 16 으로
```

- $32 \times 7 \times 7$ 이 들어오는 자리에 첫 인자 16 으로 만든 층
- 에러가 날까 · 난다면 무엇이 다르다고 할까

일부러 틀려 보기 ②

```
nn.ConvTranspose2d(32, 16, 3, stride=2, padding=1), nn.ReLU(),  
nn.ConvTranspose2d(16, 1, 3, stride=2, padding=1) # output_padding=1 을 빠뜨림
```

- 채널은 맞게 두고 `output_padding=1` 만 빠뜨림
- 쉬기 전 칸 세기: $7 \rightarrow 13 \rightarrow 25$
- 이번에는 에러가 날까

실행 — '8-0. 흔한 실수 두 개'

- 첫 줄: 실수 ① 결과
- 아래 두 줄: 실수 ② 를 지난 모양

실수 ① — 채널 에러 메시지 읽기

```
Given transposed=1, weight of size [16, 16, 3, 3],  
expected input[1, 32, 7, 7] to have 16 channels,  
but got 32 channels instead
```

- 앞부분 `Given transposed=1, weight of size ...` 는 읽지 않아도 됨
- `expected input[1, 32, 7, 7]` 의 둘째 자리(1번 자리) 32 = 들어온 채널 · `to`
`have 16 channels` = 총 첫 숫자로 적은 수 → 둘이 달라서 에러

실수 ② — 에러 없이 크기만 틀림

자리	찍힌 모양	맞게 쓴 층
ConvTranspose2d 뒤	(1, 16, 13, 13)	16×14×14
ConvTranspose2d 뒤	(1, 1, 25, 25)	1×28×28

- 쉬기 전 칸 세기와 같은 크기
- 에러 없이 끝까지 지나감 · 찍힌 모양과 주석을 대 봐야 잡힘

크기를 맞추는 순서

1. 치기 전: 총마다 나올 크기를 표로 적음
2. 치면서: 줄 끝에 크기 주석
3. 실행: 한 장을 총마다 통과시켜 찍힌 모양을 주석과 대조

'8-1.' 학습 — 학습 이미지 20,000장 · 3 에폭

```
Xsub = Xtr[:20000]  
for epoch in range(3):
```

- 학습 이미지와 에폭을 AE 보다 줄였음
- 평가는 10,000장 전체

루프는 오전과 같음

```
opt = torch.optim.Adam(cae.parameters(), lr=1e-3)
...
    loss = F.mse_loss(cae(x), x)
```

- 옵티마이저는 `cae` 의 파라미터로 새로 만들
- 모델에 넣는 것도 `x` · 정답도 `x`

ConvAE 의 복원은?

AE: 60,000 images $\times$ 10 epochs = 600,000 looks



ConvAE: 20,000 $\times$ 3 = 60,000 looks



ConvAE saw about 1/10 as many images during training

- 통로는 넓게: 입력보다 넓음
- 층도 다름: Linear 대신 Conv2d · ConvTranspose2d
- AE 보다 원본에 가까울까, 멀까 · 이유 하나

실행 — 도는 동안

- 에폭마다 한 줄씩 찍힘

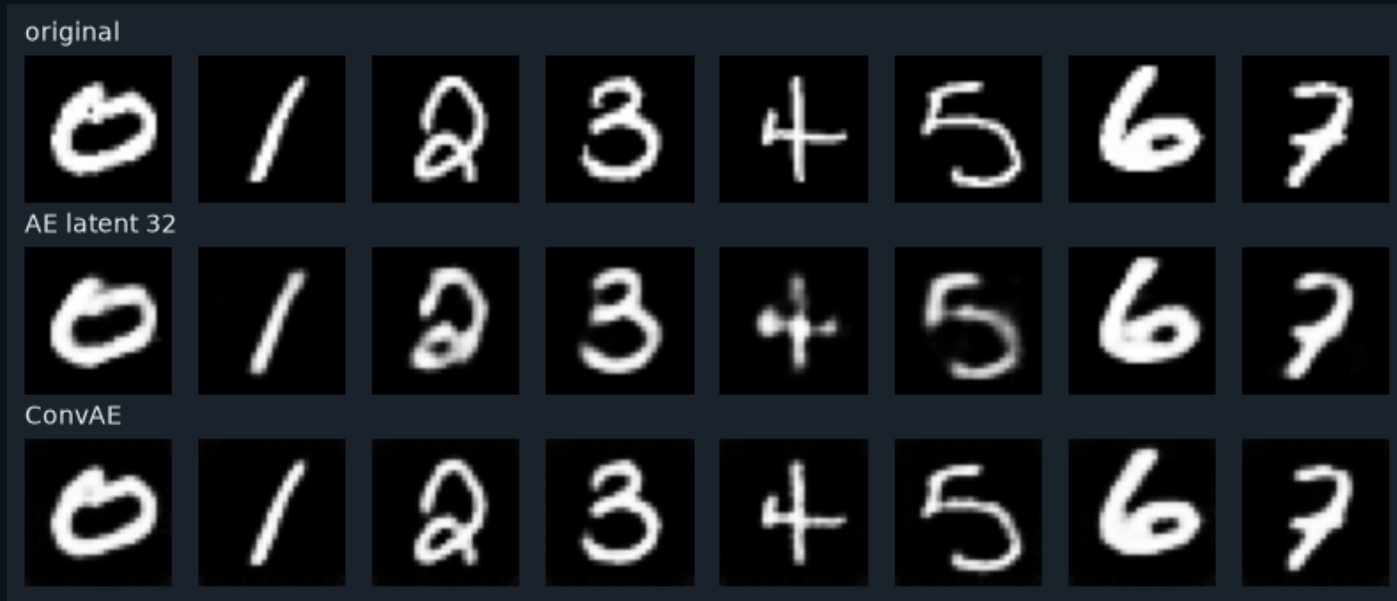
에폭마다 찍힌 줄

- 첫 에폭: 4.1초 · 학습 이미지 20,000장 한 바퀴
- loss: 마지막 묶음 하나(20,000장을 128장씩 나누고 남은 32장)에서 전 값 · 비교는 그림을 본 뒤 '8-3.' 셀로

'8-2.' 나란히 보기

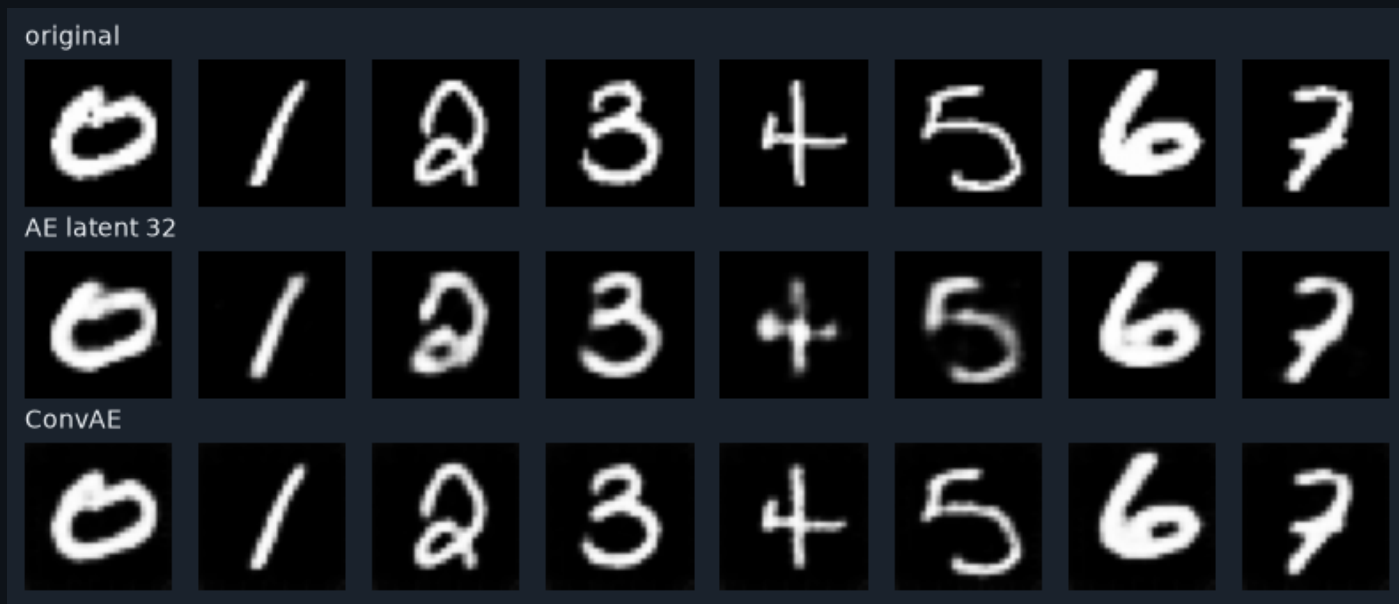
- 위에서부터 original · AE latent 32 · ConvAE

가운데 줄 — AE latent 32



- 오전 after 10 epochs 줄과 같은 모델 · 같은 여덟 장 · 같은 그림
- 0 · 1 · 3 · 6: 원본과 거의 같음 · 테두리가 번짐
- 2: 아래 고리가 뭉쳐 두꺼워짐 · 4: 획이 끊겨 십자 모양 · 5: 아래 곡선이 흐림 · 7: 윗가로획 끝이 말림

아래 줄 — ConvAE



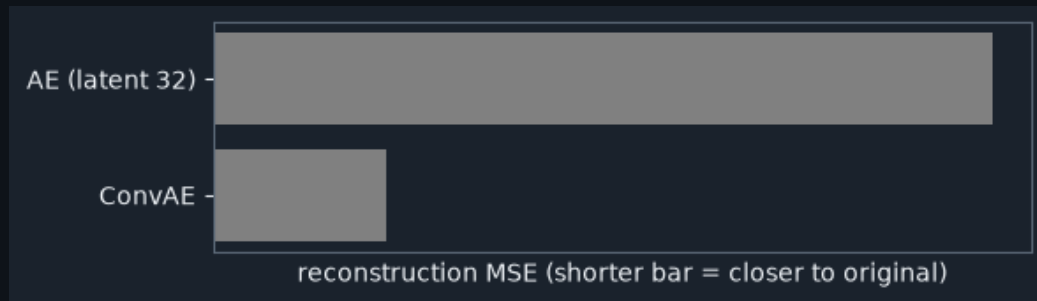
- 0 ~ 7 모두 원본과 거의 같음
- 4: 원본처럼 가는 세로획과 가로획
- 5: 윗가로획과 아래 곡선이 또렷함
- 이 크기 그림에서는 원본과 차이를 찾기 어려움

실행 — '8-3. 숫자로 확인'

- 두 모델의 복원 MSE 를 한 줄씩 찍고 막대 그림으로 나란히

숫자로 확인 — 평가 이미지 10,000장

모델	학습	복원 MSE
AE(잠재 32)	60,000장 · 10 에폭	0.0081
ConvAE	20,000장 · 3 에폭	0.0018



- 막대가 짧을수록 원본에 가까움 · ConvAE 막대가 훨씬 짧음

그럼 ConvAE 가 더 좋은 모델?

- 오차가 작고 그림도 원본에 가까움 · 이번 한 번의 실행
- 더 좋은 모델이라고 하면 될까 · 이유 하나

오차만으로는 말할 수 없음

- 가운데 통로 값 1,568개 · 입력 값 784개보다 넓음
- 입력을 거의 그대로 넘길 여지가 있는 통로
- 오차가 작다는 것만으로 "중요한 것을 골라 남기는 법을 더 잘 배웠다"고 말할 수 없음

두 모델은 조건도 다름

	AE(잠재 32)	ConvAE
학습 이미지	60,000장	20,000장
에폭	10	3
가운데 값 개수	32	1,568
줄이고 늘리는 층	Linear	Conv2d · ConvTranspose2d

- 각각 한 번씩 실행한 결과

남겨 두는 질문

통로가 입력보다 넓은 모델은 어디에 쓸모가 있을까?

- 내일: 모델에 넣는 것을 바꿔 봄

아침에 던진 질문으로 돌아가기

- 아침에 던진 질문: 통로에 무엇을 남길까
- 크기 맞추기는 그 질문을 이미지 모양 그대로 실험하려는 준비
- 이번 ConvAE: 가운데가 넓어도 되돌림 · 무엇을 남겼는지는 아직 모름

미션 5 로 가져갈 것

- 모델 설계: 인코더 `Conv2d` · 디코더 `ConvTranspose2d` 로 만든 CNN 오토인코더
- 미션의 'CNN 오토인코더' = 오늘 만든 Conv 오토인코더
- 층마다 텐서 크기를 주석으로 · 찍힌 모양과 대조
- 문서 이미지는 28×28 이 아님: 크기가 달라지면 층마다 다시 따라감

이 시간에 본 것

- Conv2d 로 줄이고 ConvTranspose2d 로 되돌린 Conv 오토인코더
- 채널이 틀리면 바로 에러 · 크기가 틀리면 주석과 대 봐야 잡힘
- 세로 · 가로 줄여도 채널 늘리면 통로는 넓어질 수 있음
- 오차 비교는 통로 크기 · 학습 조건과 같이 읽음

오늘 한 것

정답이 따로 없는 과제에서 출발

오늘 만든 두 모델

AE (Linear)

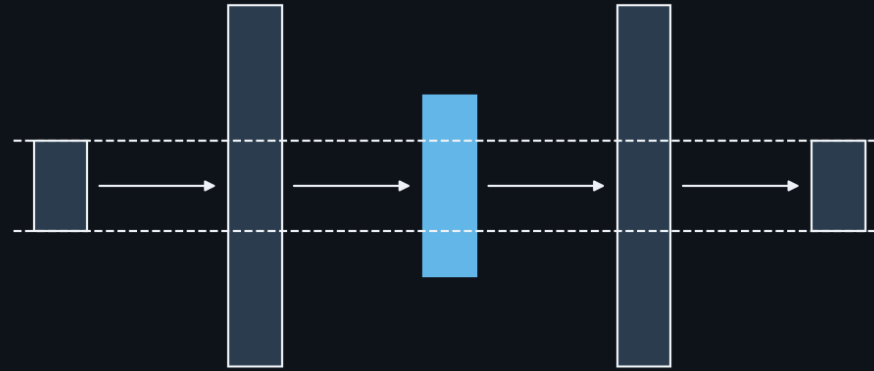


784 128 **32** 128 784

values per layer, same scale · dashed = input 784

ConvAE (Conv)

1×28×28 16×14×14 **32×7×7** 16×14×14 1×28×28



784 3,136 **1,568** 3,136 784

- 가운데 통로: AE 값 32개 · ConvAE $32 \times 7 \times 7 =$ 값 1,568개 · 점선 = 입력 784
- 복원 MSE: AE 0.0081 · ConvAE 0.0018

통로를 좁히면

- 잠재 2 · 8 · 32 · 128 비교 · 같은 구조와 같은 학습
- 잠재 2: 흐려지고 모양이 다른 숫자처럼 보이는 복원도 나옴 · 복원 MSE 도 넷 중 가장 큼
- 몇 개가 정답이라는 결론은 없음 · 통로 크기는 고르는 것

잠재 두 개를 좌표로 찍으면

- 점 하나 = 평가 이미지 한 장
- 색은 학습이 끝난 뒤에 입힌 라벨
- 찍힌 곳 안과 멀리 바깥에서 디코더가 내놓는 모습이 달랐음

크기는 적으면서 맞춤

```
ConvTranspose2d k3 s2 p1          32 → 63  
+ output_padding=1               32 → 64  
ConvTranspose2d k4 s2 p1          32 → 64  
세로 · 가로가 다른 250×536 → 두 번 줄였다 두 번 늘리면 → 252×536
```

- 총마다 크기를 주석으로 적으면 어긋나는 곳이 바로 보임

오늘의 순서 네 줄

1. 입력과 정답을 적음 — 오늘은 둘 다 같은 이미지
2. 가운데 통로의 크기를 정하고 바꿔 봄
3. 총마다 크기를 적으며 맞춤
4. 복원을 원본과 나란히 놓고 읽음 · 숫자와 그림을 같이 봄

미션 5 와 이어지는 것

- 오늘 쓴 것: 입력과 정답의 짝 · 학습 루프 · Conv 인코더와 ConvTranspose 디코더 · 크기 주석 · 나란히 읽기
- 아직 안 한 것: 망가진 입력으로 학습하기 · 원본과 얼마나 다른지 재는 점수(이름만: RMSE · PSNR · 내일)

나가기 전 질문 네 개

1. 오늘 오토인코더의 입력과 정답은 무엇이었나
2. 가운데 통로를 좁히면 복원에서 무엇이 달라졌나
3. 모델 출력 크기가 입력과 다르면 무엇부터 확인할까
4. 내일 하나만 바꾼다면(통로 크기 · 에폭 · 채널 · 층 종류 중) · 무엇을 보고 판단할까(그림 → MSE)

내일 첫 질문

입력이 더러우면, 오늘 만든 모델은 어떻게 할까